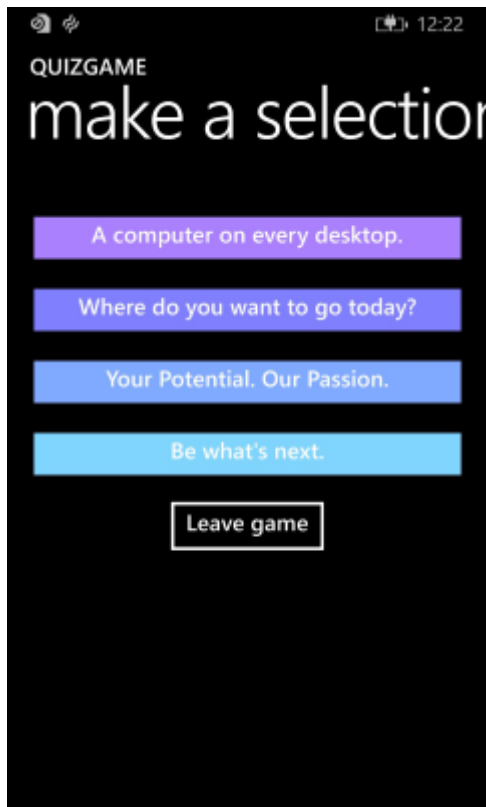




The QuizGame client app running on Windows Phone

A walkthrough of QuizGame in use

This is a short hypothetical account of the app in use, but it provides useful info should you want to try out the app for yourself over your wireless network.

A fun quiz game is taking place in a bar. There's a big TV in the bar that everyone can see. The quizmaster has a PC whose output is being shown on the TV. That PC has "the host app" running on it. Anyone who wants to take part in the quiz just needs to install "the client app" on their phone or Surface.

The host app is in lobby mode, and on the big TV, it's advertising that it's ready for client apps to connect. Joan launches the client app on her mobile device. She types her name into the **Player name** text box and taps **Join game**. The host app acknowledges that Joan has joined by displaying her name, and Joan's client app indicates that it's waiting for the game to begin. Next, Maxwell goes through those same steps on his mobile device.

The quizmaster clicks **Start game** and the host app shows a question and the possible answers (it also shows a list of the joined players in normal fontweight, colored gray). Simultaneously, the answers appear displayed on buttons on joined client devices. Joan taps the button with the answer "1975" on it whereupon all her buttons become disabled. On the host app, Joan's name is painted green (and becomes bold) in

acknowledgment of the receipt of her answer. Maxwell answers, also. The quizmaster, noting that all players' names are green, clicks **Next question**.

Questions continue to be asked and answered in this same cycle. When the last question is being shown on the host app, **Show results** is the content of the button, and not **Next question**. When **Show results** is clicked, the results are shown. Clicking **Return to lobby** returns to the beginning of the game lifecycle with the exception that joined players remain joined. But, going back to the lobby gives new players a chance to join, and even a convenient time for joined players to leave (although a joined player can leave at any time by tapping **Leave game**).

Local test mode

To try out the app and its interactions on a single PC instead of distributed across devices, you can build the host app in local test mode. This mode completely bypasses use of the network. Instead, the UI of the host app displays the host portion to the left of the window and, to the right, two copies of the client app UI stacked vertically (note that, in this version, the local test mode UI is fixed for a PC display; it does not adapt to small devices). These segments of UI, all in the same app, communicate with one another via a mock client communicator, which simulates the interactions that would otherwise take place over the network.

To activate local test mode, define **LOCALTESTMODEON** (in project properties) as a conditional compilation symbol, and rebuild.

Porting to a Windows 10 project

QuizGame has the following pieces.

- P2PHelper. This is a portable class library that contains the peer-to-peer networking logic.
- QuizGame.Windows. This is the project that builds the app package for the host app, which targets Windows 8.1.
- QuizGame.WindowsPhone. This is the project that builds the app package for the client app, which targets Windows Phone 8.1.
- QuizGame.Shared. This is the project that contains source code, markup files, and other assets and resources, that are used by both of the other two projects.

For this case study, we have the usual options described in [If you have a Universal 8.1 app](#) with respect to what devices to support.

Based on those options, we'll port QuizGame.Windows to a new Windows 10 project called QuizGameHost. And, we'll port QuizGame.WindowsPhone to a new Windows 10 project called QuizGameClient. These projects will target the universal device family, so they will run on any device. And, we'll keep the QuizGame.Shared source files, etc, in their own folder, and we'll link those shared files into the two new projects. Just like before, we'll keep everything in one solution and we'll name it QuizGame10.

The QuizGame10 solution

- Create a new solution (**New Project > Other Project Types > Visual Studio Solutions**) and name it QuizGame10.

P2PHelper

- In the solution, create a new Windows 10 class library project (**New Project > Windows Universal > Class Library (Windows Universal)**) and name it P2PHelper.
- Delete Class1.cs from the new project.
- Copy P2PSession.cs, P2PSessionClient.cs, and P2PSessionHost.cs into the new project's folder and include the copied files in the new project.
- The project will build without needing further changes.

Shared files

- Copy the folders Common, Model, View, and ViewModel from \QuizGame.Shared\ to \QuizGame10\.
- Common, Model, View, and ViewModel are what we'll mean when we refer to the shared folders on disk.

QuizGameHost

- Create a new Windows 10 app project (**Add > New Project > Windows Universal > Blank Application (Windows Universal)**) and name it QuizGameHost.
- Add a reference to P2PHelper (**Add Reference > Projects > Solution > P2PHelper**).
- In **Solution Explorer**, create a new folder for each of the shared folders on disk. In turn, right-click each folder you just created and click **Add > Existing Item** and navigate up a folder. Open the appropriate shared folder, select all files, and then click **Add As Link**.
- Copy MainPage.xaml from \QuizGame.Windows\ to \QuizGameHost\ and change the namespace to QuizGameHost.
- Copy App.xaml from \QuizGame.Shared\ to \QuizGameHost\ and change the namespace to QuizGameHost.

- Instead of overwriting app.xaml.cs, we'll keep the version in the new project and just make one targeted change to it to support local test mode. In app.xaml.cs, replace this line of code:

CSharp

```
rootFrame.Navigate(typeof(MainPage), e.Arguments);
```

with this:

CSharp

```
#if LOCALTESTMODEON
    rootFrame.Navigate(typeof(TestView), e.Arguments);
#else
    rootFrame.Navigate(typeof(MainPage), e.Arguments);
#endif
```

- In **Properties > Build > conditional compilation symbols**, add LOCALTESTMODEON.
- You'll now be able to go back to the code you added to app.xaml.cs and resolve the TestView type.
- In package.appxmanifest, change the capability name from internetClient to internetClientServer.

QuizGameClient

- Create a new Windows 10 app project (**Add > New Project > Windows Universal > Blank Application (Windows Universal)**) and name it QuizGameClient.
- Add a reference to P2PHelper (**Add Reference > Projects > Solution > P2PHelper**).
- In **Solution Explorer**, create a new folder for each of the shared folders on disk. In turn, right-click each folder you just created and click **Add > Existing Item** and navigate up a folder. Open the appropriate shared folder, select all files, and then click **Add As Link**.
- Copy MainPage.xaml from \QuizGame.WindowsPhone\ to \QuizGameClient\ and change the namespace to QuizGameClient.
- Copy App.xaml from \QuizGame.Shared\ to \QuizGameClient\ and change the namespace to QuizGameClient.
- In package.appxmanifest, change the capability name from internetClient to internetClientServer.

You'll now be able to build and run.

Adaptive UI

The QuizGameHost Windows 10 app looks fine when the app is running in a wide window (which is only possible on a device with a large screen). When the app's window is narrow, though (which happens on a small device, and can also happen on a large device), the UI is squashed so much that it's unreadable.

We can use the adaptive Visual State Manager feature to remedy this, as we explained in [Case study: Bookstore2](#). First, set properties on visual elements so that, by default, the UI is laid out in the narrow state. All of these changes take place in `\View\HostView.xaml`.

- In the main **Grid**, change the **Height** of the first **RowDefinition** from "140" to "Auto".
- On the **Grid** that contains the **TextBlock** named `pageTitle`, set `x:Name="pageTitleGrid"` and `Height="60"`. These first two steps are so that we can effectively control the height of that **RowDefinition** via a setter in a visual state.
- On `pageTitle`, set `Margin="-30,0,0,0"`.
- On the **Grid** indicated by the comment `<!-- Content -->`, set `x:Name="contentGrid"` and `Margin="-18,12,0,0"`.
- On the **TextBlock** immediately above the comment `<!-- Options -->`, set `Margin="0,0,0,24"`.
- In the default **TextBlock** style (the first resource in the file), change the **FontSize** setter's value to "15".
- In `OptionContentControlStyle`, change the **FontSize** setter's value to "20". This step and the previous one will give us a good type ramp that will work well on all devices. These are much more flexible sizes than the "30" we were using for the Windows 8.1 app.
- Finally, add the appropriate Visual State Manager markup to the root **Grid**.

XML

```
<VisualStateManager.VisualStateGroups>
  <VisualStateGroup>
    <VisualState x:Name="WideState">
      <VisualState.StateTriggers>
        <AdaptiveTrigger MinWindowWidth="548"/>
      </VisualState.StateTriggers>
      <VisualState.Setters>
        <Setter Target="pageTitleGrid.Height" Value="140"/>
        <Setter Target="pageTitle.Margin" Value="0,0,30,40"/>
        <Setter Target="contentGrid.Margin" Value="40,40,0,0"/>
      </VisualState.Setters>
    </VisualState>
  </VisualStateGroup>
</VisualStateManager.VisualStateGroups>
```

Universal styling

You'll notice that in Windows 10, the buttons don't have the same touch-target padding in their template. Two small changes will remedy that. First, add this markup to app.xaml in both QuizGameHost and QuizGameClient.

XML

```
<Style TargetType="Button">
    <Setter Property="Margin" Value="12"/>
</Style>
```

And second, add this setter to `OptionButtonStyle` in `\View\ClientView.xaml`.

XML

```
<Setter Property="Margin" Value="6"/>
```

With that last tweak, the app will behave and look just the same as it did before the port, with the additional value that it will now run everywhere.

Conclusion

The app that we ported in this case study was a relatively complex one involving several projects, a class library, and quite a large amount of code and user interface. Even so, the port was straightforward. Some of the ease of porting is directly attributable to the similarity between the Windows 10 developer platform and the Windows 8.1 and Windows Phone 8.1 platforms. Some is due to the way the original app was designed to keep the models, the view models, and the views separate.

Move from Windows Phone Silverlight to UWP

Article • 10/20/2022

If you're a developer with a Windows Phone Silverlight app, then you can make great use of your skill set and your source code in the move to Windows 10. With Windows 10, you can create a Universal Windows Platform (UWP) app, which is a single app package that your customers can install onto every kind of device. For more background on Windows 10, UWP apps, and the concepts of adaptive code and adaptive UI that we'll mention in this porting guide, see the [Guide to Universal Windows Platform \(UWP\) apps](#).

When you port your Windows Phone Silverlight app to a Windows 10 app, you'll be able to catch up on the mobile features that were [introduced in Windows Phone 8.1](#), and go far beyond them to use the Universal Windows Platform (UWP) whose app model and UI framework are universal across all Windows 10 devices. That makes it possible to support PCs, tablets, phones, and a large number of other kinds of devices, from one code base and with one app package. And that will multiply your app's potential audience and create new possibilities with shared data, purchased consumables, and so on. For more info on new features, see [What's new for developers in Windows 10](#).

If you choose to, the Windows Phone Silverlight version of your app and the Windows 10 version of it can both be available to customers at the same time.

Note This guide is designed to help you port your Windows Phone Silverlight app to Windows 10 manually. In addition to using the information in this guide to port your app, you can try the developer preview of **Mobilize.NET's Silverlight Bridge** to help automate the porting process. This tool analyzes your app's source code and converts references to Windows Phone Silverlight controls and APIs to their UWP counterparts. Because this tool is still in developer preview, it does not yet handle all conversion scenarios. However, most developers should be able to save some time and effort by starting with this tool. To try the developer preview, visit [Mobilize.NET's website](#).

XAML and .NET, or HTML?

Windows Phone Silverlight has a XAML UI framework based on Silverlight 4.0, and you program against a version of the .NET Framework and a small subset of Windows Runtime APIs. Since you used Extensible Application Markup Language (XAML) in your Windows Phone Silverlight app, it's likely that XAML will be your choice for your Windows 10 version because most of your knowledge and experience will transfer, as

will much of your source code and the software patterns you use. Even your UI markup and design can port over readily. You will find the managed APIs, the XAML markup, the UI framework, and the tooling all reassuringly familiar, and you can use C++, C#, or Visual Basic along with XAML in a UWP app. You may be surprised at how relatively easy the process is, even if there is a challenge or two along the way.

See [Roadmap for Universal Windows Platform \(UWP\) apps using C# or Visual Basic](#).

Note Windows 10 supports much more of the .NET Framework than a Windows Phone Store app does. For example, Windows 10 has several `System.ServiceModel.*` namespaces as well as `System.Net`, `System.Net.NetworkInformation`, and `System.Net.Sockets`. So, now is a great time to port your Windows Phone Silverlight and have your .NET code just compile and work on the new platform. See [Namespace and class mappings](#). Another great reason to recompile your existing .NET source code into a Windows 10 app is that you will benefit from .NET Native, which is an ahead-of-time compilation technology that converts MSIL into natively-runnable machine code. .NET Native apps start faster, use less memory, and use less battery than their MSIL counterparts.

This porting guide will focus on XAML but, alternatively, you can build a functionally equivalent app—calling many of the same Windows Runtime APIs—using JavaScript, Cascading Style Sheets (CSS), and HTML5 along with the Windows Library for JavaScript. Although the Windows Runtime UI frameworks of XAML and HTML are different from one another, whichever one you choose will work universally across the full range of Windows devices.

Targeting the universal or the mobile device family

One option you have is to port your app to an app that targets the universal device family. In this case, the app can be installed onto the widest range of devices. If your app calls APIs that are implemented only in the mobile device family, then you can guard those calls with adaptive code. Alternatively, you can choose to port your app to an app that targets the mobile device family in which case you don't need to write adaptive code.

Adapting your app to multiple form factors

The option you choose from the previous section will determine the range of devices that your app or apps will run on, and that may well be a very wide range of devices.

Even limiting your app to the mobile device family still leaves you with a wide range of screen sizes to support. So, since your app will be running on form factors that it didn't formerly support, test your UI on those form factors and make any change necessary so that your UI adapts appropriately on each. You can think of this as a post-porting task, or a porting stretch-goal, and there is an example of it in practice in the [Bookstore2](#) case study.

Approaching porting layer-by-layer

- **View.** The view (together with the view model) makes up your app's UI. Ideally, the view consists of markup bound to observable properties of a view model. Another pattern (common and convenient, but only in the short term) is for imperative code in a code-behind file to directly manipulate UI elements. In either case, much of your UI markup and design—and even imperative code that manipulates UI elements—will be straightforward to port.
- **View models and data models.** Even if you don't formally embrace separation-of-concerns patterns (such as MVVM), there is inevitably code present in your app that performs the function of view model and data model. View model code makes use of types in the UI framework namespaces. Both view model and data model code also use non-visual operating system and .NET APIs (including APIs for data-access). And the vast majority of those are [available to a UWP app](#), so you can expect to be able to port much of this code without change. Remember, though: a view model is a model, or *abstraction*, of a view. A view model provides the state and behavior of UI, while the view itself provides the visuals. For this reason, any UI you adapt to the different form factors that the UWP allows you to run on will likely need corresponding view model changes. For networking and calling cloud services, you typically have the option between using .NET or Windows Runtime APIs. For the factors involved in making that decision, see [Cloud services, networking, and databases](#).
- **Cloud services.** It's likely that some of your app (perhaps a great deal of it) runs in the cloud in the form of services. The part of the app running on the client device connects to those. This is the part of a distributed app most likely to remain unchanged when porting the client part. If you don't already have one, a good cloud services option for your UWP app is [Microsoft Azure Mobile Services](#) [↗](#), which provides powerful back-end components that Universal Windows apps can call for services ranging from simple notifications for live tiles updates up to the kind of heavy-lifting scalability a server farm can provide.

Before or during the porting, consider whether your app could be improved by refactoring it so that code with a similar purpose is gathered together in layers and not scattered arbitrarily. Factoring your UWP app into layers like those described above

makes it easier for you to make your app correct, to test it, and then subsequently to read and maintain it. You can make functionality more reusable—and avoid some issues of UI API differences between platforms—by following the Model-View-ViewModel (MVVM) pattern. This pattern keeps the data, business, and UI parts of your app separate from one another. Even within the UI it can keep state and behavior separate, and separately testable, from the visuals. With MVVM, you can write your data and business logic once and use it on all devices no matter the UI. It's likely that you'll be able to re-use much of the view model and view parts across devices, too.

One or two exceptions to the rule

As you read this porting guide, you can refer to [Namespace and class mappings](#). Fairly straightforward mapping is the general rule, and the namespace and class mappings table describes any exceptions.

At the feature level, the good news is that there's very little that's not supported in the UWP. Most of your skill set and source code translates very well over to UWP apps, as you'll read in the rest of this porting guide. But, here are the few Windows Phone Silverlight features that you may have used for which there is no UWP equivalent.

Feature for which there is no UWP equivalent	Windows Phone Silverlight documentation for the feature
Microsoft XNA. In general, Microsoft DirectX using C++ is the replacement. See Developing games and DirectX and XAML interop .	XNA Framework Class Library
Lens apps	Lenses for Windows Phone 8

Topic	Description
Namespace and class mappings	This topic provides a comprehensive mapping of Windows Phone Silverlight APIs to their UWP equivalents.
Porting the project	You begin the porting process by creating a new Windows 10 project in Visual Studio and copying your files into it.

Topic	Description
Troubleshooting	We highly recommend reading to the end of this porting guide, but we also understand that you're eager to forge ahead and get to the stage where your project builds and runs. To that end, you can make temporary progress by commenting or stubbing out any non-essential code, and then returning to pay off that debt later. The table of troubleshooting symptoms and remedies in this topic may be helpful to you at this stage, although it's not a substitute for reading the next few topics. You can always refer back to the table as you progress through the later topics.
Porting XAML and UI	The practice of defining UI in the form of declarative XAML markup translates extremely well from Windows Phone Silverlight to UWP apps. You'll find that large sections of your markup are compatible once you've updated system Resource key references, changed some element type names, and changed "clr-namespace" to "using".
Porting for I/O, device, and app model	Code that integrates with the device itself and its sensors involves input from, and output to, the user. It can also involve processing data. But, this code is not generally thought of as either the UI layer or the data layer. This code includes integration with the vibration controller, accelerometer, gyroscope, microphone and speaker (which intersect with speech recognition and synthesis), (geo)location, and input modalities such as touch, mouse, keyboard, and pen.
Porting business and data layers	Behind your UI are your business and data layers. The code in these layers calls operating system and .NET Framework APIs (for example, background processing, location, the camera, the file system, network, and other data access). The vast majority of those are available to a UWP app , so you can expect to be able to port much of this code without change.
Porting for form factor and UX	Windows apps share a common look-and-feel across PCs, mobile devices, and many other kinds of devices. The user interface, input, and interaction patterns are very similar, and a user moving between devices will welcome the familiar experience.
Case study: Bookstore1	This topic presents a case study of porting a very simple Windows Phone Silverlight app to a Windows 10 UWP app. With Windows 10, you can create a single app package that your customers can install onto a wide range of devices, and that's what we'll do in this case study.
Case study: Bookstore2	This case study—which builds on the info given in Bookstore1 —begins with a Windows Phone Silverlight app that displays grouped data in a LongListSelector . In the view model, each instance of the class Author represents the group of the books written by that author, and in the LongListSelector , we can either view the list of books grouped by author or we can zoom out to see a jump list of authors.

Related topics

Documentation

- [What's new for developers in Windows 10](#)
- [Guide to Universal Windows Platform \(UWP\) apps](#)
- [Roadmap for Universal Windows Platform \(UWP\) apps using C# or Visual Basic](#)
- [What's next for Windows Phone 8 developers](#)

Magazine articles

- [Visual Studio Magazine: Windows Phone 8.1: A Giant Leap Forward for Convergence](#) 

Presentations

- The Story of Bringing Nokia Music from Windows Phone to Windows 8

Windows Phone Silverlight to UWP API mappings

Article • 10/20/2022

This topic provides a comprehensive mapping of Windows Phone Silverlight APIs to their Universal Windows Platform (UWP) equivalents. There is generally not a one-to-one mapping of functionality, though: either platform may have more or less functionality than its counterpart in a namespace or class.

The mapping table will help you when you're working in a UWP project and you're re-using source code from a Windows Phone Silverlight project. There are differences in the names of namespaces and classes (including UI controls) between the two platforms. In many cases, it's as easy as changing a namespace name and then your code will compile. Sometimes, a class or API name has changed as well as the namespace name. Other times, the mapping takes a bit more work, and in rare cases requires a change in approach.

****How to use the table:** ** First, search for the name of the class you're using. Classes are listed whenever the mapping is more complicated than simply changing the namespace name. If your class is not listed, then the mapping is just a namespace change. So, find your class's namespace name and you'll find the equivalent UWP namespace name. Your class will be in that namespace. If your namespace is not listed, then its name has not changed.

Note Windows 10 supports much more of the .NET Framework than a Windows Phone Store app does. For example, Windows 10 has several `System.ServiceModel.*` namespaces as well as `System.Net`, `System.Net.NetworkInformation`, and `System.Net.Sockets`. Also, in a Windows 10 app, you will benefit from .NET Native, which is an ahead-of-time compilation technology that converts MSIL into natively-runable machine code. .NET Native apps start faster, use less memory, and use less battery than their MSIL counterparts.

Windows Phone Silverlight	Windows Runtime
Advertising	
<code>Microsoft.Advertising.Mobile.UI.AdControl</code> class	AdControl class
Alarms, reminders, and background agents	
<code>Microsoft.Phone.BackgroundAgent</code> class	BackgroundTaskBuilder class

Windows Phone Silverlight	Windows Runtime
Microsoft.Phone.Scheduler namespace	Windows.ApplicationModel.Background namespace
Microsoft.Phone.Scheduler.Alarm class	BackgroundTaskBuilder and ToastNotificationManager classes
Microsoft.Phone.Scheduler.PeriodicTask, ScheduledAction, ScheduledActionService, ScheduledTask , ScheduledTaskAgent classes	BackgroundTaskBuilder class
Microsoft.Phone.Scheduler.Reminder class	BackgroundTaskBuilder and ToastNotificationManager classes
Microsoft.Phone.PictureDecoder class	BitmapDecoder class
Microsoft.Phone.BackgroundAudio namespace	Windows.Media.Playback namespace
Microsoft.Phone.BackgroundTransfer namespace	Windows.Networking.BackgroundTransfer namespace
App model and environment	
System.AppDomain class	No direct equivalent. See Application , CoreApplication , classes
System.Environment class	No direct equivalent
System.ComponentModel.Annotations class	No direct equivalent
System.ComponentModel.BackgroundWorker class	ThreadPool class
System.ComponentModel.DesignerProperties class	DesignMode class
System.Threading.Thread, System.Threading.ThreadPool classes	ThreadPool class
(ST = System.Threading) ST.Thread.MemoryBarrier method	(ST = System.Threading) ST.Interlocked.MemoryBarrier method
(ST = System.Threading) ST.Thread.ManagedThreadId property	(S = System) S.Environment.ManagedThreadId property
System.Threading.Timer class	ThreadPoolTimer class
(SWT = System.Windows.Threading) SWT.Dispatcher class	CoreDispatcher class

Windows Phone Silverlight	Windows Runtime
(SWT = <code>System.Windows.Threading</code>) <code>SWT.DispatcherTimer</code> class	DispatcherTimer class
Blend for Visual Studio	
(MEDC = <code>Microsoft.Expression.Drawing.Core</code>) <code>MEDC.GeometryHelper</code> class	No direct equivalent
<code>Microsoft.Expression.Interactivity</code> namespace	Microsoft.Xaml.Interactivity namespace
<code>Microsoft.Expression.Interactivity.Core</code> namespace	Microsoft.Xaml.Interactions.Core namespace
(MEIC = <code>Microsoft.Expression.Interactivity.Core</code>) <code>MEIC.ExtendedVisualStateManager</code> class	No direct equivalent
<code>Microsoft.Expression.Interactivity.Input</code> namespace	No direct equivalent
<code>Microsoft.Expression.Interactivity.Media</code> namespace	Microsoft.Xaml.Interactions.Media namespace
<code>Microsoft.Expression.Shapes</code> namespace	No direct equivalent
(MI = <code>Microsoft.Internal</code>) <code>MI.IManagedFrameworkInternalHelper</code> interface	No direct equivalent
Contact and calendar data	
<code>Microsoft.Phone.UserData</code> namespace	Windows.ApplicationModel.Contacts , Windows.ApplicationModel.Appointments namespaces
(MPU = <code>Microsoft.Phone.UserData</code>) <code>MPU.Account</code> , <code>ContactAddress</code> , <code>ContactCompanyInformation</code> , <code>ContactEmailAddress</code> , <code>ContactPhoneNumber</code> classes	Contact class
(MPU = <code>Microsoft.Phone.UserData</code>) <code>MPU.Appointments</code> class	AppointmentCalendar class
(MPU = <code>Microsoft.Phone.UserData</code>) <code>MPU.Contacts</code> class	ContactStore class
Controls and UI infrastructure	

Windows Phone Silverlight	Windows Runtime
ControlTiltEffect.TiltEffect class	Animations from the Windows Runtime animation library are built into the default Styles of the common controls. See Animation .
Microsoft.Phone.Controls namespace	Windows.UI.Xaml.Controls namespace
(MPC = Microsoft.Phone.Controls) MPC.ContextMenu class	PopupMenu class
(MPC = Microsoft.Phone.Controls) MPC.DatePickerPage class	DatePickerFlyout class
(MPC = Microsoft.Phone.Controls) MPC.GestureListener class	GestureRecognizer class
(MPC = Microsoft.Phone.Controls) MPC.LongListSelector class	SemanticZoom class
(MPC = Microsoft.Phone.Controls) MPC.ObscuredEventArgs class	SystemProtection , WindowActivatedEventArgs classes
(MPC = Microsoft.Phone.Controls) MPC.Panorama class	Hub class
(MPC = Microsoft.Phone.Controls) MPC.PhoneApplicationFrame , (SWN = System.Windows.Navigation) SWN.NavigationService classes	Frame class
(MPC = Microsoft.Phone.Controls) MPC.PhoneApplicationPage class	Page class
(MPC = Microsoft.Phone.Controls) MPC.TiltEffect class	PointerDownThemeAnimation class
(MPC = Microsoft.Phone.Controls) MPC.TimePickerPage class	TimePickerFlyout class
(MPC = Microsoft.Phone.Controls) MPC.WebBrowser class	WebView class
(MPC = Microsoft.Phone.Controls) MPC.WebBrowserExtensions class	No direct equivalent
(MPC = Microsoft.Phone.Controls) MPC.WrapPanel class	No direct equivalent for general layout purposes. ItemsWrapGrid and WrapGrid can be used in the items panel template of an items control.

Windows Phone Silverlight	Windows Runtime
(MPD = Microsoft.Phone.Data) MPD.Linq namespace	No direct equivalent
(MPD = Microsoft.Phone.Data) MPD.Linq.Mapping namespace	No direct equivalent
Microsoft.Phone.Globalization namespace	No direct equivalent
(MPI = Microsoft.Phone.Info) MPI.DeviceExtendedProperties, DeviceStatus classes	EasClientDeviceInformation , MemoryManager classes. For more details, see Device status .
(MPI = Microsoft.Phone.Info) MPI.MediaCapabilities class	No direct equivalent
(MPI = Microsoft.Phone.Info) MPI.UserExtendedProperties class	AdvertisingManager class
System.Windows namespace	Windows.UI.Xaml namespace
System.Windows.Automation namespace	Windows.UI.Xaml.Automation namespace
System.Windows.Controls, System.Windows.Input namespaces	Windows.UI.Core , Windows.UI.Input , Windows.UI.Xaml.Controls namespaces
System.Windows.Controls.DrawingSurface, DrawingSurfaceBackgroundGrid classes	SwapChainPanel class
System.Windows.Controls.RichTextBox class	RichEditBox class
System.Windows.Controls.WrapPanel class	No direct equivalent for general layout purposes. ItemsWrapGrid and WrapGrid can be used in the items panel template of an items control.
System.Windows.Controls.Primitives namespace	Windows.UI.Xaml.Controls.Primitives namespace
System.Windows.Controls.Shapes namespace	Windows.UI.Xaml.Controls.Shapes namespace
System.Windows.Data namespace	Windows.UI.Xaml.Data namespace
System.Windows.Documents namespace	Windows.UI.Xaml.Documents namespace
System.Windows.Ink namespace	No direct equivalent
System.Windows.Markup namespace	Windows.UI.Xaml.Markup namespace
System.Windows.Navigation namespace	Windows.UI.Xaml.Navigation namespace

Windows Phone Silverlight	Windows Runtime
System.Windows.UIElement.Tap event, EventHandler<GestureEventArgs> delegate	Tapped event, TappedEventHandler delegate
Data and services	
System.Data.Linq.DataContext class	No direct equivalent
System.Data.Linq.Mapping.ColumnAttribute class	No direct equivalent
System.Data.Linq.SqlClient.SqlHelpers class	No direct equivalent
Devices	
Microsoft.Devices, Microsoft.Devices.Sensors namespaces	Windows.Devices.Enumeration , Windows.Devices.Enumeration.Pnp , Windows.Devices.Input , Windows.Devices.Sensors namespaces
Microsoft.Devices.Camera, Microsoft.Devices.PhotoCamera classes	MediaCapture class. Also, CameraCaptureUI class (Windows only).
Microsoft.Devices.CameraButtons class	HardwareButtons class
Microsoft.Devices.CameraVideoBrushExtensions class	CaptureElement class
Microsoft.Devices.Environment class	No direct equivalent. As a workaround, use conditional compilation and define a custom symbol. Or you may be able to engineer a workaround using the IsAttached property.
Microsoft.Devices.MediaHistory class	No direct equivalent
Microsoft.Devices.VibrateController class	VibrationDevice class
Microsoft.Devices.Radio.FMRadio class	No direct equivalent
Microsoft.Devices.Sensors.Accelerometer, Compass classes	In the Windows.Devices.Sensors namespace
Microsoft.Devices.Sensors.Gyroscope class	Gyrometer class
Microsoft.Devices.Sensors.Motion class	Inclinometer class
Globalization	
System.Globalization namespace	Windows.Globalization namespace

Windows Phone Silverlight	Windows Runtime
(ST = <code>System.Threading</code>) <code>ST.Thread.CurrentCulture</code> property	(SG = <code>System.Globalization</code>) <code>S.CultureInfo.CurrentCulture</code> property
(ST = <code>System.Threading</code>) <code>ST.Thread.CurrentUICulture</code> property	(SG = <code>System.Globalization</code>) <code>S.CultureInfo.CurrentUICulture</code> property
Graphics and animation	
<code>Microsoft.Xna.Framework.*</code> namespaces, XNA Framework Class Library , Content Pipeline Class Library	No direct equivalent. In general, use Microsoft DirectX with C++. See Developing games and DirectX and XAML interop .
<code>Microsoft.Xna.Framework.Audio.Microphone</code> class	MediaCapture class
<code>Microsoft.Xna.Framework.Audio.SoundEffect</code> class	MediaElement class
<code>Microsoft.Xna.Framework.GamerServices</code> namespace	(WPS = <code>Windows.Phone.System</code>) WPS.UserProfile.GameServices.Core namespace
<code>Microsoft.Xna.Framework.GamerServices.Guide</code> class	No direct equivalent
<code>Microsoft.Xna.Framework.Input.GamePad</code> class	HardwareButtons class
<code>Microsoft.Xna.Framework.Input.Touch.TouchPanel</code> class	GestureRecognizer class
(MXFM = <code>Microsoft.Xna.Framework.Media</code>) <code>MXFM.MediaLibrary</code> , <code>MXFM.PhoneExtensions.MediaLibraryExtensions</code> classes	KnownFolders class
<code>Microsoft.Xna.Framework.Media.MediaQueue</code> class	SystemMediaTransportControls class
<code>Microsoft.Xna.Framework.Media.Playlist</code> class	BackgroundMediaPlayer class
<code>System.Windows.Media</code> namespace	Windows.UI.Xaml.Media namespace
<code>System.Windows.Media.RadialGradientBrush</code> class	No direct equivalent. See Media and graphics .
<code>System.Windows.Media.Animation</code> namespace	Windows.UI.Xaml.Media.Animation namespace
<code>System.Windows.Media.Effects</code> namespace	No direct equivalent

Windows Phone Silverlight	Windows Runtime
System.Windows.Media.Imaging namespace	Windows.UI.Xaml.Media.Imaging namespace
System.Windows.Media.Media3D namespace	Windows.UI.Xaml.Media.Media3D namespace
System.Windows.Shapes namespace	Windows.UI.Xaml.Shapes namespace
Launchers and Choosers	
Microsoft.Phone.Tasks.AddressChooserTask, EmailAddressChooserTask, PhoneNumberChooserTask classes	ContactPicker class
Microsoft.Phone.Tasks.AddWalletItemTask, AddWalletItemResult classes	Windows.ApplicationModel.Wallet namespace
Microsoft.Phone.Tasks.BingMapsDirectionsTask, BingMapsTask classes	No direct equivalent
Microsoft.Phone.Tasks.CameraCaptureTask class	MediaCapture class. Also, CameraCaptureUI class (Windows only).
Microsoft.Phone.Tasks.MarketplaceDetailTask	CurrentApp class (RequestAppPurchaseAsync method)
Microsoft.Phone.Tasks.ConnectionSettingsTask, MarketplaceHubTask, MarketplaceReviewTask, MarketplaceSearchTask, MediaPlayerLauncher, SearchTask, SmsComposeTask, WebBrowserTask classes	Launcher class
Microsoft.Phone.Tasks.EmailComposeTask class	EmailMessage class
Microsoft.Phone.Tasks.GameInviteTask class	No direct equivalent
Microsoft.Phone.Tasks.MapDownloaderTask, MapsDirectionsTask, MapsTask, MapUpdaterTask classes	No direct equivalent
Microsoft.Phone.Tasks.PhoneCallTask class	PhoneCallManager class
Microsoft.Phone.Tasks.PhotoChooserTask class	FileOpenPicker class
Microsoft.Phone.Tasks.SaveAppointmentTask class	AppointmentManager class
Microsoft.Phone.Tasks.SaveContactTask, SaveEmailAddressTask, SavePhoneNumberTask classes	StoredContact class (Windows Phone only)

Windows Phone Silverlight	Windows Runtime
Microsoft.Phone.Tasks.SaveRingtoneTask class	No direct equivalent
Microsoft.Phone.Tasks.ShareLinkTask, ShareMediaTask, ShareStatusTask classes	DataPackage class
Location	
System.Device.Location namespace	Windows.Devices.Geolocation namespace
System.Device.GeoCoordinateWatcher class	Geolocator class
Maps	
Microsoft.Phone.Maps namespaces	Windows.Services.Maps namespace
Microsoft.Phone.Maps.Controls namespace	Windows.UI.Xaml.Controls.Maps namespace
Microsoft.Phone.Maps.Controls.Map class	MapControl class
Microsoft.Phone.Maps.Services namespace	Windows.Services.Maps namespace
Microsoft.Phone.Maps.Services.GeocodeQuery, ReverseGeocodeQuery classes	MapLocationFinder class
System.Device.Location.GeoCoordinate class	Geopoint class
Microsoft.Phone.Maps.Services.Route class	MapRoute class
Microsoft.Phone.Maps.Services.RouteQuery class	MapRouteFinder class
Monetization	
Microsoft.Phone.Marketplace namespace	Windows.ApplicationModel.Store namespace
Media	
Microsoft.Phone.Media namespace	MediaElement class
Networking	
(MPNN = Microsoft.Phone.Net.NetworkInformation) MPNN.DeviceNetworkInformation class	Hostname , NetworkInformation classes
(MPNN = Microsoft.Phone.Net.NetworkInformation) MPNN.NetworkInterface class	NetworkInformation class

Windows Phone Silverlight	Windows Runtime
(MPNN = <code>Microsoft.Phone.Net.NetworkInformation</code>) <code>MPNN.NetworkInterfaceInfo</code> class	ConnectionProfile class
(MPNN = <code>Microsoft.Phone.Net.NetworkInformation</code>) <code>MPNN.NetworkInterfaceList</code> class	NetworkInformation class
(MPNN = <code>Microsoft.Phone.Net.NetworkInformation</code>) <code>MPNN.SocketExtensions</code> class	No direct equivalent
(MPNN = <code>Microsoft.Phone.Net.NetworkInformation</code>) <code>MPNN.WebRequestExtensions</code> class	No direct equivalent
<code>Microsoft.Phone.Networking.Voip</code> namespace	No direct equivalent
<code>System.Net.CookieCollection</code> class	Still supported, but some properties are missing (for example, <code>IsReadOnly</code>)
<code>System.Net.DownloadProgressChangedEventArgs</code> class, and similar classes related to <code>System.Net.WebClient</code>	HttpClient class (or System.Net.Http.HttpClient). Derive from System.Net.Http.StreamContent to measure progress.
<code>System.Net.DnsEndPoint</code> , <code>IPAddress</code> classes	These classes are still supported, but some properties are missing. Alternatively, port to the HostName class.
<code>System.Net.HttpUtility</code> class	HtmlFormatHelper class
<code>System.Net.HttpWebRequest</code> class	Partial support, but the recommended, forward-looking alternative is the HttpClient class (or System.Net.Http.HttpClient). These APIs use System.Net.Http.HttpRequestMessage to represent an HTTP request.
<code>System.Net.HttpWebResponse</code> class	Still supported, but use <code>Dispose()</code> instead of <code>Close()</code> . But, the recommended, forward-looking alternative is the HttpClient class (or System.Net.Http.HttpClient). These APIs use System.Net.Http.HttpResponseMessage to represent an HTTP response.
(SNN = <code>System.Net.NetworkInformation</code>) <code>SNN.NetworkChange</code> class	Still supported, except for the constructor.

Windows Phone Silverlight	Windows Runtime
<code>System.Net.OpenReadCompletedEventArgs</code> class, and similar classes related to <code>System.Net.WebClient</code>	HttpClient class (or System.Net.Http.HttpClient)
<code>System.Net.Sockets.Socket</code> class	Still supported, but use <code>Dispose()</code> instead of <code>Close()</code> . Alternatively, port to the StreamSocket class.
<code>System.Net.Sockets.SocketException</code> class	Still supported, but use the <code>SocketErrorCode</code> property instead of <code>ErrorCode</code> .
<code>System.Net.Sockets.UdpAnySourceMulticastClient</code> , <code>UdpSingleSourceMulticastClient</code> classes	DatagramSocket class
<code>System.Net.UploadProgressChangedEventArgs</code> class, and similar classes related to <code>System.Net.WebClient</code>	HttpClient class (or System.Net.Http.HttpClient)
<code>System.Net.WebClient</code> class	HttpClient class (or System.Net.Http.HttpClient)
<code>System.Net.WebRequest</code> class	Partial support (a different set of properties), but the recommended, forward-looking alternative is the HttpClient class (or System.Net.Http.HttpClient). These APIs use System.Net.Http.HttpRequestMessage to represent an HTTP request.
<code>System.Net.WebResponse</code> class	Still supported, but use <code>Dispose()</code> instead of <code>Close()</code> . But, the recommended, forward-looking alternative is the HttpClient class (or System.Net.Http.HttpClient). These APIs use System.Net.Http.HttpResponseMessage to represent an HTTP response.
(SN = <code>System.Net</code>) <code>SN.WriteStreamClosedEventArgs</code> class	HttpClient class (or System.Net.Http.HttpClient)
(SN = <code>System.Net</code>) <code>SN.WriteStreamClosedEventHandler</code> class	HttpClient class (or System.Net.Http.HttpClient)
<code>System.UriFormatException</code> class	<code>System.FormatException</code> class
Notifications	

Windows Phone Silverlight	Windows Runtime
MPN = Microsoft.Phone.Notification namespace	Windows.UI.Notifications , Windows.Networking.PushNotifications namespaces
MPN = Microsoft.Phone.Notification MPN.HttpNotification class	TileNotification class
MPN = Microsoft.Phone.Notification MPN.HttpNotificationChannel class	PushNotificationChannel class
Programming	
System namespace	Windows.Foundation namespace
System.Diagnostics.StackFrame , StackTrace classes	No direct equivalent
System.Diagnostics namespace	Windows.Foundation.Diagnostics namespace
System.ICloneable interface	A custom method that returns the appropriate type.
System.Reflection.Emit.ILGenerator class	No direct equivalent
Reactive Extensions	
Microsoft.Phone.Reactive namespace	No direct equivalent
Reflection	
System.Type class	System.Reflection.TypeInfo class. See Reflection in the .NET Framework for UWP apps .
Resources	
System.Resources.ResourceManager class	(WA = Windows.ApplicationModel) WA.Resources.Core and WA.Resources namespaces, ResourceManager class. See Creating and retrieving resources in Windows Runtime apps .
Secure Element	
(MPS = Microsoft.Phone.SecureElement) MPS.SecureElementChannel , MPS.SecureElementSession classes	SmartCardConnection class
(MPS = Microsoft.Phone.SecureElement) MPS.SecureElementReader class	SmartCardReader class

Windows Phone Silverlight	Windows Runtime
Security	
(SSC = System.Security.Cryptography) SSC.Aes, SSC.RSA classes	CryptographicEngine class
(SSC = System.Security.Cryptography) SSC.HMACSHA256, SSC.SHA256 classes	HashAlgorithmProvider class
(SSC = System.Security.Cryptography) SSC.ProtectedData class	DataProtectionProvider class
(SSC = System.Security.Cryptography) SSC.RandomNumberGenerator class	CryptographicBuffer class
(SSC = System.Security.Cryptography) SSC.X509Certificates.X509Certificate class	CertificateEnrollmentManager class
Shell	
(MPSh = Microsoft.Phone.Shell) MPSh.ApplicationBar class	CommandBar class
(MPSh = Microsoft.Phone.Shell) MPSh.ApplicationBarIconButton class	AppBarButton class (when used inside the PrimaryCommands property)
(MPSh = Microsoft.Phone.Shell) MPSh.ApplicationBarMenuItem class	AppBarButton class (when used inside the SecondaryCommands property)
(MPSh = Microsoft.Phone.Shell) MPSh.CycleTileData, MPSh.FlipTileData, MPSh.IconicTileData, MPSh.ShellTileData, MPSh.StandardTileData classes	TileTemplateType class
(MPSh = Microsoft.Phone.Shell) MPSh.PhoneApplicationService class	CoreApplication , DisplayRequest classes
(MPSh = Microsoft.Phone.Shell) MPSh.ProgressIndicator class	StatusBarProgressIndicator class
(MPSh = Microsoft.Phone.Shell) MPSh.ShellTile class	SecondaryTile class
(MPSh = Microsoft.Phone.Shell) MPSh.ShellTileSchedule class	TileUpdater class
(MPSh = Microsoft.Phone.Shell) MPSh.ShellToast class	ToastNotificationManager class
(MPSh = Microsoft.Phone.Shell) MPSh.SystemTray class	StatusBar class

Windows Phone Silverlight	Windows Runtime
Storage and I/O	
Microsoft.Phone.Storage.ExternalStorage, ExternalStorageDevice, ExternalStorageFile, ExternalStorageFolder classes	KnownFolders class
System.IO namespace	Windows.Storage , Windows.Storage.Streams namespaces
System.IO.Directory class	StorageFolder class
System.IO.File class	StorageFile and PathIO classes
(SII = System.IO.IsolatedStorage) SII.IsolatedStorageFile class	ApplicationData.LocalFolder property
(SII = System.IO.IsolatedStorage) SII.IsolatedStorageSettings class	ApplicationData.LocalSettings property
System.IO.Stream class	Still supported, but use ReadAsync() and WriteAsync() instead of BeginRead()/EndRead() and BeginWrite()/EndWrite().
Wallet	
Microsoft.Phone.Wallet namespace	Windows.ApplicationModel.Wallet namespace
Xml	
(SX = System.Xml)	SX.XmlConvert.ToDateTime method
(SX = System.Xml)	SX.XmlConvert.ToDateTimeOffset method

The next topic is [Porting the project](#).

Porting Windows Phone Silverlight projects to UWP projects

Article • 10/20/2022

The previous topic was [Namespace and class mappings](#).

You begin the porting process by creating a new Windows 10 project in Visual Studio and copying your files into it.

Create the project and copy files to it

1. Launch Microsoft Visual Studio 2015 and create a new Blank Application (Windows Universal) project. For more info, see [Jumpstart your Windows Runtime 8.x app using templates \(C#, C++, Visual Basic\)](#). Your new project builds an app package (an appx file) that will run on all device families.
2. In your Windows Phone Silverlight app project, identify all the source code files and visual asset files that you want to reuse. Using File Explorer, copy data models, view models, visual assets, Resource Dictionaries, folder structure, and anything else that you wish to re-use, to your new project. Copy or create sub-folders on disk as necessary.
3. Copy views (for example, MainPage.xaml and MainPage.xaml.cs) into the new project node, too. Again, create new sub-folders as necessary, and remove the existing views from the project. But, before you over-write or remove a view that Visual Studio generated, keep a copy because it may be useful to refer to it later. The first phase of porting a Windows Phone Silverlight app focuses on getting it to look good and work well on one device family. Later, you'll turn your attention to making sure the views adapt themselves well to all form factors, and optionally to adding any adaptive code to get the most from a particular device family.
4. In **Solution Explorer**, make sure **Show All Files** is toggled on. Select the files that you copied, right-click them, and click **Include In Project**. This will automatically include their containing folders. You can then toggle **Show All Files** off if you like. An alternative workflow, if you prefer, is to use the **Add Existing Item** command, having created any necessary sub-folders in the Visual Studio **Solution Explorer**. Double-check that your visual assets have **Build Action** set to **Content** and **Copy to Output Directory** set to **Do not copy**.
5. The differences in namespace and class names will generate lots of build errors at this stage. For example, if you open the views that Visual Studio generated, you'll see that they are of type [Page](#), and not **PhoneApplicationPage**. There are lots of XAML markup and imperative code differences that the following topics in this

porting guide cover in detail. But, you'll make fast progress just following these general steps: change "clr-namespace" to "using" in your namespace prefix declarations in XAML markup; use the [Namespace and class mappings](#) topic and Visual Studio's **Find and Replace** command to make bulk changes to your source code (for example, replace "System.Windows" with "Windows.UI.Xaml"); and in the imperative code editor in Visual Studio use the **Resolve** and **Organize Usings** commands on the context menu for more targeted changes.

Extension SDKs

Most of the Universal Windows Platform (UWP) APIs your ported app will call are implemented in the set of APIs known as the universal device family. But, some are implemented in extension SDKs, and Visual Studio only recognizes APIs that are implemented by your app's target device family or by any extension SDKs that you have referenced.

If you get compile errors about namespaces or types or members that could not be found, then this is likely to be the cause. Open the API's topic in the API reference documentation and navigate to the Requirements section: that will tell you what the implementing device family is. If that's not your target device family, then to make the API available to your project, you will need a reference to the extension SDK for that device family.

Click **Project > Add Reference > Windows Universal > Extensions** and select the appropriate extension SDK. For example, if the APIs you want to call are available only in the mobile device family, and they were introduced in version 10.0.x.y, then select **Windows Mobile Extensions for the UWP**.

That will add the following reference to your project file:

XML

```
<ItemGroup>
  <SDKReference Include="WindowsMobile, Version=10.0.x.y">
    <Name>Windows Mobile Extensions for the UWP</Name>
  </SDKReference>
</ItemGroup>
```

The name and version number match the folders in the installed location of your SDK. For example, the above information matches this folder name:

```
\Program Files (x86)\Windows Kits\10\Extension SDKs\WindowsMobile\10.0.x.y
```

Unless your app targets the device family that implements the API, you'll need to use the [ApiInformation](#) class to test for the presence of the API before you call it (this is called adaptive code). This condition will then be evaluated wherever your app runs, but it will only evaluate to true on devices where the API is present and therefore available to call. Only use extension SDKs and adaptive code after first checking whether a universal API exists. Some examples are given in the section below.

Also, see [App package manifest](#).

Maximizing markup and code reuse

You will find that refactoring a little, and/or adding adaptive code (which is explained below), will allow you to maximize the markup and code that works across all device families. Here are more details.

- Files that are common to all device families need no special consideration. Those files will be used by the app on all the device families that it runs on. This includes XAML markup files, imperative source code files, and asset files.
- It is possible for your app to detect the device family that it is running on and navigate to a view that has been designed specifically for that device family. For more details, see [Detecting the platform your app is running on](#).
- A similar technique that you may find useful if there is no alternative is to give a markup file or **ResourceDictionary** file (or the folder that contains the file) a special name such that it is automatically loaded at runtime only when your app runs on a particular device family. This technique is illustrated in the [Bookstore1](#) case study.
- To use features that are not available on all device families (for example, printers, scanners, or the camera button) you can write adaptive code. See the third example in [Conditional compilation and adaptive code](#) in this topic.
- If you want to support both Windows Phone Silverlight and Windows 10, then you may be able to share source code files between projects. Here's how: in Visual Studio, right-click the project in **Solution Explorer**, select **Add Existing Item**, select the files to share, and then click **Add As Link**. Store your source code files in a common folder on the file system where the projects that link to them can see them, and don't forget to add them to source control. If you can factor your imperative source code so that most, if not all, of a file will work on both platforms, then you don't need to have two copies of it. You can wrap any platform-specific logic in the file inside conditional compilation directives where possible, or run-time conditions where necessary. See the next section below, and [C# Preprocessor Directives](#).
- For reuse at the binary level, rather than the source code level, there are Portable Class Libraries, which support the subset of .NET APIs that are available in

Windows Phone Silverlight as well as the subset for Windows 10 apps (.NET Core). Portable Class Library assemblies are binary compatible with these .NET platforms and more. Use Visual Studio to create a project that targets a Portable Class Library. See [Cross-Platform Development with the Portable Class Library](#).

Conditional compilation and adaptive code

If you want to support both Windows Phone Silverlight and Windows 10 in a single code file then you can do that. If you look in your Windows 10 project at the project properties pages, you'll see that the project defines `WINDOWS_UAP` as a conditional compilation symbol. In general, you can use the following logic to perform conditional compilation.

C#

```
#if WINDOWS_UAP
    // Code that you want to compile into the Windows 10/11 app.
#else
    // Code that you want to compile into the Windows Phone Silverlight app.
#endif // WINDOWS_UAP
```

If you have code that you've been sharing between a Windows Phone Silverlight app and a Windows Runtime 8.x app, then you may already have source code with logic like this:

C#

```
#if NETFX_CORE
    // Code that you want to compile into the Windows Runtime 8.x app.
#else
    // Code that you want to compile into the Windows Phone Silverlight app.
#endif // NETFX_CORE
```

If so, and if you now want to support Windows 10 in addition, then you can do that, too.

C#

```
#if WINDOWS_UAP
    // Code that you want to compile into the Windows 10/11 app.
#else
    #if NETFX_CORE
        // Code that you want to compile into the Windows Runtime 8.x app.
    #else
        // Code that you want to compile into the Windows Phone Silverlight app.
    #endif // NETFX_CORE
#endif // WINDOWS_UAP
```

You may have used conditional compilation to limit handling of the hardware back button to Windows Phone. In Windows 10, the back button event is a universal concept. Back buttons implemented in hardware or in software will all raise the [BackRequested](#) event, so that's the one to handle.

C#

```
Windows.UI.Core.SystemNavigationManager.GetForCurrentView().BackRequested +=  
    this.ViewModelLocator_BackRequested;  
  
...  
  
private void ViewModelLocator_BackRequested(object sender,  
Windows.UI.Core.BackRequestedEventArgs e)  
{  
    // Handle the event.  
}
```

You may have used conditional compilation to limit handling of the hardware camera button to Windows Phone. In Windows 10, the hardware camera button is a concept particular to the mobile device family. Because one app package will be running on all devices, we change our compile-time condition into a run-time condition using what is known as adaptive code. To do that, we use the [ApiInformation](#) class to query at run-time for the presence of the [HardwareButtons](#) class. **HardwareButtons** is defined in the mobile extension SDK, so we'll need to add a reference to that SDK to our project for this code to compile. Note, though, that the handler will only be executed on a device that implements the types defined in the mobile extension SDK, and that's the mobile device family. So, the following code is careful only to use features that are present, although it achieves it in a different way from conditional compilation.

C#

```
// Note: Cache the value instead of querying it more than once.  
bool isHardwareButtonsAPIPresent =  
Windows.Foundation.Metadata.ApiInformation.IsTypePresent  
    ("Windows.Phone.UI.Input.HardwareButtons");  
  
if (isHardwareButtonsAPIPresent)  
{  
    Windows.Phone.UI.Input.HardwareButtons.CameraPressed +=  
        this.HardwareButtons_CameraPressed;  
}  
  
...
```

```
private void HardwareButtons_CameraPressed(object sender,
Windows.Phone.UI.Input.CameraEventArgs e)
{
    // Handle the event.
}
```

Also, see [Detecting the platform your app is running on](#).

The app package manifest

The settings in your project (including any extension SDKs references) determine the API surface area that your app can call. But, your app package manifest is what determines the actual set of devices that your customers can install your app onto from the Store. For more info, see Examples in [TargetDeviceFamily](#).

It's worth knowing how to edit the app package manifest, because the topics that follow talk about using it for various declarations, capabilities, and other settings that some features need. You can use the Visual Studio app package manifest editor to edit it. If the **Solution Explorer** is not shown, choose it from the **View** menu. Double-click **Package.appxmanifest**. This opens the manifest editor window. Select the appropriate tab to make changes and then save the changes. You may want to ensure that the **pm:PhoneIdentity** element in the ported app manifest matches what is in the app manifest of the app you're porting (for full details, see the [pm:PhoneIdentity](#) topic).

See [Package manifest schema reference for Windows 10](#).

The next topic is [Troubleshooting](#).

Troubleshooting porting Windows Phone Silverlight to UWP

Article • 10/20/2022

The previous topic was [Porting the project](#).

We highly recommend reading to the end of this porting guide, but we also understand that you're eager to forge ahead and get to the stage where your project builds and runs. To that end, you can make temporary progress by commenting or stubbing out any non-essential code, and then returning to pay off that debt later. The table of troubleshooting symptoms and remedies in this topic may be helpful to you at this stage, although it's not a substitute for reading the next few topics. You can always refer back to the table as you progress through the later topics.

Tracking down issues

XAML parse exceptions can be difficult to diagnose, particularly if there are no meaningful error messages within the exception. Make sure that the debugger is configured to catch first-chance exceptions (to try and catch the parsing exception early on). You may be able to inspect the exception variable in the debugger to determine whether the HRESULT or message has any useful information. Also, check Visual Studio's output window for error messages output by the XAML parser.

If your app terminates and all you know is that an unhandled exception was thrown during XAML markup parsing, then that could be the result of a reference to a missing resource (that is, a resource whose key exists for Windows Phone Silverlight apps but not for Windows 10 apps, such as some system **TextBlock** Style keys). Or, it could be an exception thrown inside a **UserControl**, a custom control, or a custom layout panel.

A last resort is a binary split. Remove about half of the markup from a Page and re-run the app. You will then know whether the error is somewhere inside the half you removed (which you should now restore in any case) or in the half you did *not* remove. Repeat the process by splitting the half that contains the error, and so on, until you've zeroed in on the issue.

TargetPlatformVersion

This section explains what to do if, on opening a Windows 10 project in Visual Studio, you see the message "Visual Studio update required. One or more projects require a

platform SDK <version> that is either not installed or is included as part of a future update to Visual Studio."

- First, determine the version number of the SDK for Windows 10 that you have installed. Navigate to **C:\Program Files (x86)\Windows Kits\10\Include\<versionfoldername>** and make a note of <versionfoldername>, which will be in quad notation, "Major.Minor.Build.Revision".
- Open your project file for edit and find the `TargetPlatformVersion` and `TargetPlatformMinVersion` elements. Edit them to look like this, replacing <versionfoldername> with the quad notation version number you found on disk:

XML

```
<TargetPlatformVersion><versionfoldername></TargetPlatformVersion>
<TargetPlatformMinVersion><versionfoldername></TargetPlatformMinVersion>
```

Troubleshooting symptoms and remedies

The remedy information in the table is intended to give you enough info to unblock yourself. You'll find further details about each of these issues as you read through later topics.

Symptom	Remedy
The XAML parser or compiler gives the error <i>"The name "<typename>" does not exist in the namespace [...]."</i>	If <typename> is a custom type then, in your namespace prefix declarations in XAML markup, change "clr-namespace" to "using", and remove any assembly tokens. For platform types, this means that the type doesn't apply to the Universal Windows Platform (UWP), so find the equivalent and update your markup. Examples you might encounter right away are <code>phone:PhoneApplicationPage</code> and <code>shell:SystemTray.IsVisible</code> .
The XAML parser or compiler gives the error <i>"The member "<membername>" is not recognized or is not accessible." or "The property "<propertyname>" was not found in type [...]."</i>	These errors will begin to show up after you've ported some type names, such as the root Page . The member or property doesn't apply to the UWP, so find the equivalent and update your markup. Examples you might encounter right away are <code>SupportedOrientations</code> and <code>Orientation</code> .

Symptom	Remedy
The XAML parser or compiler gives the error <i>"The attachable property [...] was not found [...]"</i> or <i>"Unknown attachable member [...]"</i> .	This is likely to be caused by the type rather than the attached property; in which case, you will already have an error for the type and this error will go away once you fix that. Examples you might encounter right away are <code>phone:PhoneApplicationPage.Resources</code> and <code>phone:PhoneApplicationPage.DataContext</code> .
The XAML parser or compiler, or a runtime exception, gives the error <i>"The resource "<resourcekey>" could not be resolved."</i>	The resource key doesn't apply to Universal Windows Platform (UWP) apps. Find the correct equivalent resource and update your markup. Examples you might encounter right away are system TextBlock Style keys such as <code>PhoneTextNormalStyle</code> .
The C# compiler gives the error <i>"The type or namespace name '<name>' could not be found [...]"</i> or <i>"The type or namespace name '<name>' does not exist in the namespace [...]"</i> or <i>"The type or namespace name '<name>' does not exist in the current context"</i> .	This is likely to mean that the compiler doesn't yet know the correct UWP namespace for a type. Use Visual Studio's Resolve command to fix that. If the API is not in the set of APIs known as the universal device family (in other words, the API is implemented in an extension SDK), then use the Extension SDKs . There may be other cases where port is less straightforward. Examples you might encounter right away are <code>DesignerProperties</code> and <code>BitmapImage</code> .
When run on the device, the app terminates, or when launched from Visual Studio, you see the error "Unable to activate Windows Runtime 8.x app [...]. The activation request failed with error 'Windows was unable to communicate with the target application. This usually indicates that the target application's process aborted. [...]"	The problem could be the imperative code running in your own Pages or in bound properties (or other types) during initialization. Or, it could be happening while parsing the XAML file about to be displayed when the app terminated (if launching from Visual Studio, that will be the startup page). Look for invalid resource keys, and/or try some of the guidance in the Tracking down issues section in this topic.
<i>XamlCompiler error WMC0055: Cannot assign text value '<your stream geometry>' into property 'Clip' of type 'RectangleGeometry'</i>	In the UWP, the type of the Microsoft DirectX and XAML C++ UWP app.
<i>XamlCompiler error WMC0001: Unknown type 'RadialGradientBrush' in XML namespace [...]</i>	The UWP doesn't have the RadialGradientBrush type. Remove the RadialGradientBrush from markup and use some other type of Microsoft DirectX and XAML C++ UWP app.

Symptom	Remedy
<i>XamlCompiler error WMC0011: Unknown member 'OpacityMask' on element '<UIElement type>'</i>	The UWP Microsoft DirectX and XAML C++ UWP app.
<i>A first chance exception of type 'System.Runtime.InteropServices.COMException' occurred in SYSTEM.NI.DLL. Additional information: The application called an interface that was marshalled for a different thread. (Exception from HRESULT: 0x8001010E (RPC_E_WRONG_THREAD)).</i>	The work you're doing needs to be done on the UI thread. Call the CoreWindow.GetForCurrentThread).
An animation is running, but it's having no effect on its target property.	Either make the animation independent, or set <code>EnableDependentAnimation="True"</code> on it. See Animation .
On opening a Windows 10 project in Visual Studio, you see the message "Visual Studio update required. One or more projects require a platform SDK <version> that is either not installed or is included as part of a future update to Visual Studio."	See the TargetPlatformVersion section in this topic.
A System.InvalidCastException is thrown when InitializeComponent is called in a xaml.cs file.	This can happen when you have more than one xaml file (at least one of which is MRT-qualified) sharing the same xaml.cs file and elements have x:Name attributes that are inconsistent between the two xaml files. Try adding the same name to the same elements in both xaml files, or omit names altogether.

The next topic is [Porting XAML and UI](#).

Porting Windows Phone Silverlight XAML and UI to UWP

Article • 10/20/2022

The previous topic was [Troubleshooting](#).

The practice of defining UI in the form of declarative XAML markup translates extremely well from Windows Phone Silverlight to Universal Windows Platform (UWP) apps. You'll find that large sections of your markup are compatible once you've updated system Resource key references, changed some element type names, and changed "clr-namespace" to "using". Much of the imperative code in your presentation layer—view models, and code that manipulates UI elements—will also be straightforward to port.

A first look at the XAML markup

The previous topic showed you how to copy your XAML and code-behind files into your new Windows 10 Visual Studio project. One of the first issues you might notice highlighted in the Visual Studio XAML designer is that the `PhoneApplicationPage` element at the root of your XAML file is not valid for a Universal Windows Platform (UWP) project. In the previous topic, you saved a copy of the XAML files that Visual Studio generated when it created the Windows 10 project. If you open that version of `MainPage.xaml`, you'll see that at the root is the type `Page`, which is in the `Windows.UI.Xaml.Controls` namespace. So, you can change all

`<phone:PhoneApplicationPage>` elements to `<Page>` (don't forget property element syntax) and you can delete the `xmlns:phone` declaration.

For a more general approach to finding the UWP type that corresponds to a Windows Phone Silverlight type, you can refer to [Namespace and class mappings](#).

XAML namespace prefix declarations

If you use instances of custom types in your views—perhaps a view model instance or a value converter—then you will have XAML namespace prefix declarations in your XAML markup. The syntax of these differs between Windows Phone Silverlight and the UWP. Here are some examples:

XML

```
xmlns:ContosoTradingCore="clr-namespace:ContosoTradingCore;assembly=ContosoTradingCore"
```

```
xmlns:ContosoTradingLocal="clr-namespace:ContosoTradingLocal"
```

Change "clr-namespace" to "using" and delete any assembly token and semi-colon (the assembly will be inferred). The result looks like this:

XML

```
xmlns:ContosoTradingCore="using:ContosoTradingCore"  
xmlns:ContosoTradingLocal="using:ContosoTradingLocal"
```

You may have a resource whose type is defined by the system:

XML

```
xmlns:System="clr-namespace:System;assembly=mscorlib"  
/* ... */  
<System:Double x:Key="FontSizeLarge">40</System:Double>
```

In the UWP, omit the "System" prefix declaration and use the (already declared) "x" prefix instead:

XML

```
<x:Double x:Key="FontSizeLarge">40</x:Double>
```

Imperative code

Your view models are one place where there's imperative code that references UI types. Another place is any code-behind files that directly manipulate UI elements. For example, you might find that a line of code like this one doesn't compile yet:

C#

```
return new BitmapImage(new Uri(this.CoverImagePath, UriKind.Relative));
```

BitmapImage is in the **System.Windows.Media.Imaging** namespace in Windows Phone Silverlight, and a using directive in the same file allows **BitmapImage** to be used without namespace qualification as in the snippet above. In a case like this, you can right-click the type name (**BitmapImage**) in Visual Studio and use the **Resolve** command on the context menu to add a new namespace directive to the file. In this case, the [Windows.UI.Xaml.Media.Imaging](#) namespace is added, which is where the type lives in the UWP. You can remove the **System.Windows.Media.Imaging** using directive, and that

will be all it takes to port code like that in the snippet above. When you're done, you'll have removed all Windows Phone Silverlight namespaces.

In simple cases like this, where you're mapping the types in an old namespace to the same types in a new one, you can use Visual Studio's **Find and Replace** command to make bulk changes to your source code. The **Resolve** command is a great way of discovering a type's new namespace. As another example, you can replace all "System.Windows" with "Windows.UI.Xaml". That will essentially port all using directives and all fully-qualified type names that refer to that namespace.

Once all the old using directives are removed and the new ones added, you can use Visual Studio's **Organize Usings** command to sort your directives and remove unused ones.

Sometimes, fixing imperative code will be as minor as changing a parameter's type. Other times, you will need to use Windows Runtime APIs instead of .NET APIs for Windows Runtime 8.x apps. To identify which APIs are supported, use the rest of this porting guide in combination with [.NET for Windows Runtime 8.x apps overview](#) and the [Windows Runtime reference](#).

And, if you just want to get to the stage where your project builds, you can comment or stub out any non-essential code. Then iterate, one issue at a time, and refer to the following topics in this section (and the previous topic: [Troubleshooting](#)), until any build and runtime issues are ironed-out and your port is complete.

Adaptive/responsive UI

Because your Windows 10 app can run on a potentially wide range of devices—each with its own screen size and resolution—you'll want to go beyond the minimal steps to port your app and you'll want to tailor your UI to look its best on those devices. You can use the adaptive Visual State Manager feature to dynamically detect window size and to change layout in response, and an example of how to do that is shown in the section [Adaptive UI](#) in the Bookstore2 case study topic.

Alarms and Reminders

Code using the **Alarm** or **Reminder** classes should be ported to use the [BackgroundTaskBuilder](#) class to create and register a background task, and display a toast at the relevant time. See [Background processing](#) and [Toasts](#).

Animation

As a preferred alternative to keyframe animations and from/to animations, the UWP animation library is available to UWP apps. These animations have been designed and tuned to run smoothly, to look great, and to make your app appear as integrated with Windows as the built-in apps do. See [Quickstart: Animating your UI using library animations](#).

If you do use keyframe animations or from/to animations in your UWP apps, then you may want to understand the distinction between independent and dependent animations that the new platform has introduced. See [Optimize animations and media](#). Animations that run on the UI thread (ones that animate layout properties, for example) are known as dependent animations, and when run on the new platform, they will have no effect unless you do one of two things. You can either re-target them to animate different properties, such as [RenderTransform](#), thereby making them independent. Or you can set `EnableDependentAnimation="True"` on the animation element in order to confirm your intention to run an animation that cannot be guaranteed to run smoothly. If you use Blend for Visual Studio to author new animations, then that property will be set for you where necessary.

Back button handling

In a Windows 10 app, you can use a single approach to handling the back button and it will work on all devices. On mobile devices, the button is provided for you as a capacitive button on the device, or as a button in the shell. On a desktop device, you add a button to your app's chrome whenever back-navigation is possible within the app, and this appears in the title bar for windowed apps or in the task bar for [Tablet mode \(Windows 10 only\)](#). The back button event is a universal concept across all device families, and buttons implemented in hardware or in software raise the same [BackRequested](#) event.

The example below works for all device families and it is good for cases where the same processing applies to all pages, and where you needn't confirm navigation (for example, to warn about unsaved changes).

C#

```
// app.xaml.cs

protected override void OnLaunched(LaunchActivatedEventArgs e)
{
    [...]

    Windows.UI.Core.SystemNavigationManager.GetForCurrentView().BackRequested +=
    App_BackRequested;
```



```

        rootFrame.Navigated += RootFrame_Navigated;
    }

    private void RootFrame_Navigated(object sender, NavigationEventArgs e)
    {
        Frame rootFrame = Window.Current.Content as Frame;

        // Note: On device families that have no title bar, setting
        // AppViewBackButtonVisibility can safely execute
        // but it will have no effect. Such device families provide a back
        // button UI for you.
        if (rootFrame.CanGoBack)
        {
            Windows.UI.Core.SystemNavigationManager.GetForCurrentView().AppViewBackButtonVisibility =
                Windows.UI.Core.AppViewBackButtonVisibility.Visible;
        }
        else
        {
            Windows.UI.Core.SystemNavigationManager.GetForCurrentView().AppViewBackButtonVisibility =
                Windows.UI.Core.AppViewBackButtonVisibility.Collapsed;
        }
    }

    private void App_BackRequested(object sender,
    Windows.UI.Core.BackRequestedEventArgs e)
    {
        Frame rootFrame = Window.Current.Content as Frame;

        if (rootFrame.CanGoBack)
        {
            rootFrame.GoBack();
        }
    }
}

```

There's also a single approach for all device families for programmatically exiting the app.

C#

```
Windows.UI.Xaml.Application.Current.Exit();
```

Binding, and compiled bindings with {x:Bind}

The topic of binding includes:

- Binding a UI element to "data" (that is, to the properties and commands of a view model)
- Binding a UI element to another UI element
- Writing a view model that is observable (that is, it raises notifications when a property value changes and when the availability of a command changes)

All of these aspects are largely still supported, but there are namespace differences. For example, `System.Windows.Data.Binding` maps to [Windows.UI.Xaml.Data.Binding](#), `System.ComponentModel.INotifyPropertyChanged` maps to [Windows.UI.Xaml.Data.INotifyPropertyChanged](#), and `System.Collections.Specialized.INotifyPropertyChanged` maps to [Windows.UI.Xaml.Interop.INotifyCollectionChanged](#).

Windows Phone Silverlight app bars and app bar buttons can't be bound like they can in a UWP app. You may have imperative code that constructs your app bar and its buttons, binds them to properties and localized strings, and handles their events. If so, you now have the option to port that imperative code by replacing it with declarative markup bound to properties and commands, and with static resource references, thus making your app incrementally safer and more maintainable. You can use Visual Studio or Blend for Visual Studio to bind and style UWP app bar buttons just like any other XAML element. Note that in a UWP app the type names you use are [CommandBar](#) and [AppBarButton](#).

The binding-related features of UWP apps currently have the following limitations:

- There is no built-in support for data-entry validation and the [IDataErrorInfo](#) and [INotifyDataErrorInfo](#) interfaces.
- The [Binding](#) class does not include the extended formatting properties available in Windows Phone Silverlight. However, you can still implement [IValueConverter](#) to provide custom formatting.
- The [IValueConverter](#) methods take language strings as parameters instead of [CultureInfo](#) objects.
- The [CollectionViewSource](#) class does not provide built-in support for sorting and filtering, and grouping works differently. For more info, see [Data binding in depth](#) and the [Data binding sample](#) [↗](#).

Although the same binding features are still largely supported, Windows 10 offers the option of a new and more performant binding mechanism called compiled bindings, which use the `{x:Bind}` markup extension. See [Data Binding: Boost Your Apps' Performance Through New Enhancements to XAML Data Binding](#), and the [x:Bind Sample](#) [↗](#).

Binding an Image to a view model

You can bind the [Image.Source](#) property to any property of a view model that's of type [ImageSource](#). Here's a typical implementation of such a property in a Windows Phone Silverlight app:

C#

```
// this.BookCoverImagePath contains a path of the form
"/Assets/CoverImages/one.png".
return new BitmapImage(new Uri(this.CoverImagePath, UriKind.Relative));
```

In a UWP app, you use the ms-appx [URI scheme](#). So that you can keep the rest of your code the same, you can use a different overload of the **System.Uri** constructor to put the ms-appx URI scheme in a base URI and append the rest of the path onto that. Like this:

C#

```
// this.BookCoverImagePath contains a path of the form
"/Assets/CoverImages/one.png".
return new BitmapImage(new Uri(new Uri("ms-appx://"),
this.CoverImagePath));
```

That way, the rest of the view model, the path values in the image path property, and the bindings in the XAML markup, can all stay exactly the same.

Controls, and control styles/templates

Windows Phone Silverlight apps use controls defined in the **Microsoft.Phone.Controls** namespace and the **System.Windows.Controls** namespace. XAML UWP apps use controls defined in the [Windows.UI.Xaml.Controls](#) namespace. The architecture and design of XAML controls in the UWP is virtually the same as Windows Phone Silverlight controls. But, some changes have been made to improve the set of available controls and to unify them with Windows apps. Here are specific examples.

Control name	Change
AppBar	The Page.TopAppBar property.
AppBarIconButton	The UWP equivalent is the Glyph property. PrimaryCommands is the content property of CommandBar. The XAML parser interprets an element's inner xml as the value of its content property.

Control name	Change
AppBarMenuItem	The UWP equivalent is the AppBarButton.Label set to the menu item text.
ContextMenu (in the Windows Phone Toolkit)	For a single selection fly-out, use Flyout .
ControlTiltEffect.TiltEffect class	Animations from the UWP animation library are built into the default Styles of the common controls. See the Animating pointer actions .
LongListSelector with grouped data	The Windows Phone Silverlight LongListSelector functions in two ways, which can be used in concert. First, it is able to display data that is grouped by a key, for example, a list of names grouped by initial letter. Second, it is able to "zoom" between two semantic views: the grouped list of items (for example, names), and a list of just the group keys themselves (for example, initial letters). With the UWP, you can display grouped data with the Guidelines for list and grid view controls .
LongListSelector with flat data	For performance reasons, in the case of very long lists, we recommended LongListSelector instead of a Windows Phone Silverlight list box even for flat, non-grouped data. In a UWP app, GridView are preferred for long lists of items whether or not the data are amenable to grouping.
Panorama	The Windows Phone Silverlight Panorama control maps to the Guidelines for hub controls in Windows Runtime 8.x apps and Guidelines for the hub control. Note that a Panorama control wraps around from the last section to the first, and its background image moves in parallax relative to the sections. Hub sections do not wrap around, and parallax is not used.
Pivot	The UWP equivalent of the Windows Phone Silverlight Pivot control is Windows.UI.Xaml.Controls.Pivot . It is available for all device families.

Note The PointerOver visual state is relevant in custom styles/templates in Windows 10 apps, but not in Windows Phone Silverlight apps. There are other reasons why your existing custom styles/templates may not be appropriate for Windows 10 apps, including system resource keys you are using, changes to the sets of visual states used, and performance improvements made to the Windows 10 default styles/templates. We recommend that you edit a fresh copy of a control's default template for Windows 10 and then re-apply your style and template customization to that.

For more info on UWP controls, see [Controls by function](#), [Controls list](#), and [Guidelines for controls](#).

Design language in Windows 10

There are some differences in design language between Windows Phone Silverlight apps and Windows 10 apps. For all the details, see [Design](#). Despite the design language changes, our design principles remain consistent: be attentive to detail but always strive for simplicity through focusing on content not chrome, fiercely reducing visual elements, and remaining authentic to the digital domain; use visual hierarchy especially with typography; design on a grid; and bring your experiences to life with fluid animations.

Localization and globalization

For localized strings, you can re-use the .resx file from your Windows Phone Silverlight project in your UWP app project. Copy the file over, add it to the project, and rename it to Resources.resw so that the lookup mechanism will find it by default. Set **Build Action** to **PRResource** and **Copy to Output Directory** to **Do not copy**. You can then use the strings in markup by specifying the **x:Uid** attribute on your XAML elements. See [Quickstart: Using string resources](#).

Windows Phone Silverlight apps use the **CultureInfo** class to help globalize an app. UWP apps use MRT (Modern Resource Technology), which enables the dynamic loading of app resources (localization, scale, and theme) both at runtime and in the Visual Studio design surface. For more information, see [Guidelines for files, data, and globalization](#).

The [ResourceContext.QualifierValues](#) topic describes how to load device family-specific resources based on the device family resource selection factor.

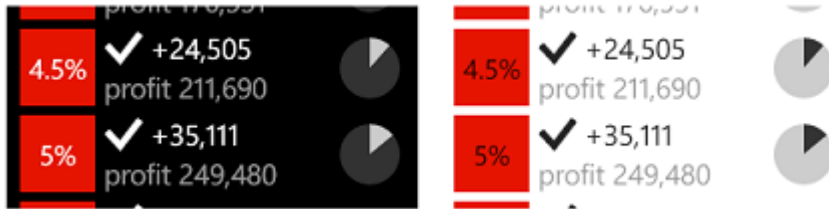
Media and graphics

As you read about UWP media and graphics, bear in mind that the Windows design principles encourage a fierce reduction of anything superfluous, including graphical complexity and clutter. Windows design is typified by clean and clear visuals, typography, and motion. If your app follows the same principles, then it will seem more like the built-in apps.

Windows Phone Silverlight has a **RadialGradientBrush** type which is not present in the UWP, although other [Brush](#) types are. In some cases, you will be able to get a similar effect with a bitmap. Note that you can [create a radial gradient brush](#) with Direct2D in a [Microsoft DirectX](#) and XAML C++ UWP.

Windows Phone Silverlight has the **System.Windows.UIElement.OpacityMask** property, but that property is not a member of the UWP [UIElement](#) type. In some cases, you will

be able to get a similar effect with a bitmap. And you can [create an opacity mask](#) with Direct2D in a [Microsoft DirectX](#) and XAML C++ UWP app. But, a common use case for **OpacityMask** is to use a single bitmap that adapts to both light and dark themes. For vector graphics, you can use theme-aware system brushes (such as the pie charts illustrated below). But, to make a theme-aware bitmap (such as the check marks illustrated below), requires a different approach.



In a Windows Phone Silverlight app, the technique is to use an alpha mask (in the form of a bitmap) as the **OpacityMask** for a **Rectangle** filled with the foreground brush:

XML

```
<Rectangle Fill="{StaticResource PhoneForegroundBrush}" Width="26"
Height="26">
    <Rectangle.OpacityMask>
        <ImageBrush ImageSource="/Assets/wpsl_check.png"/>
    </Rectangle.OpacityMask>
</Rectangle>
```

The most straightforward way to port this to a UWP app is to use a [BitmapIcon](#), like this:

XML

```
<BitmapIcon UriSource="Assets/winrt_check.png" Width="21" Height="21"/>
```

Here, winrt_check.png is an alpha mask in the form of a bitmap just as wpsl_check.png is, and it could very well be the same file. However, you may want to provide several different sizes of winrt_check.png to be used for different scaling factors. For more info on that, and for an explanation of the changes to the **Width** and **Height** values, see [View or effective pixels, viewing distance, and scale factors](#) in this topic.

A more general approach, which is appropriate if there are differences between the light and dark theme form of a bitmap, is to use two image assets—one with a dark foreground (for light theme) and one with a light foreground (for dark theme). For further details about how to name this set of bitmap assets, see [Tailor your resources for language, scale, and other qualifiers](#). Once a set of image files are correctly named, you can refer to them in the abstract, using their root name, like this:

XML

```
<Image Source="Assets/winrt_check.png" Stretch="None"/>
```

In Windows Phone Silverlight, the **UIElement.Clip** property can be any shape that you can express with a **Geometry** and is typically serialized in XAML markup in the **StreamGeometry** mini-language. In the UWP, the type of the **Clip** property is **RectangleGeometry**, so you can only clip a rectangular region. Allowing a rectangle to be defined using mini-language would be too permissive. So, to port a clipping region in markup, replace the **Clip** attribute syntax and make it into property element syntax similar to the following:

XML

```
<UIElement.Clip>
    <RectangleGeometry Rect="10 10 50 50"/>
</UIElement.Clip>
```

Note that you can [use arbitrary geometry as a mask in a layer](#) with Direct2D in a [Microsoft DirectX](#) and XAML C++ UWP app.

Navigation

When you navigate to a page in a Windows Phone Silverlight app, you use a Uniform Resource Identifier (URI) addressing scheme:

C#

```
NavigationService.Navigate(new Uri("/AnotherPage.xaml",
UriKind.Relative)/*, navigationState*/);
```

In a UWP app, you call the **Frame.Navigate** method and specify the type of the destination page (as defined by the **x:Class** attribute of the page's XAML markup definition):

C#

```
// In a page:
this.Frame.Navigate(typeof(AnotherPage)/*, parameter*/);

// In a view model, perhaps inside an ICommand implementation:
var rootFrame = Windows.UI.Xaml.Window.Current.Content as
Windows.UI.Xaml.Controls.Frame;
rootFrame.Navigate(typeof(AnotherPage)/*, parameter*/);
```

You define the startup page for a Windows Phone Silverlight app in WMApManifest.xml:

XML

```
<DefaultTask Name="_default" NavigationPage="MainPage.xaml" />
```

In a UWP app, you use imperative code to define the startup page. Here's some code from App.xaml.cs that illustrates how:

C#

```
if (!rootFrame.Navigate(typeof(MainPage), e.Arguments))
```

URI mapping and fragment navigation are URI navigation techniques, and so they are not applicable to UWP navigation, which is not based on URIs. URI mapping exists in response to the weakly-typed nature of identifying a target page with a URI string, which leads to fragility and maintainability issues should the page move to a different folder and hence to a different relative path. UWP apps use type-based navigation, which is strongly-typed and compiler-checked, and does not have the problem that URI mapping solves. The use case for fragment navigation is to pass along some context to the target page so that the page can cause a particular fragment of its content to be scrolled into view, or otherwise displayed. The same goal can be achieved by passing a navigation parameter when you call the [Navigate](#) method.

For more info, see [Navigation](#).

Resource key reference

The design language has evolved for Windows 10 and consequently certain system styles have changed, and many system resource keys have been removed or renamed. The XAML markup editor in Visual Studio highlights references to resource keys that can't be resolved. For example, the XAML markup editor will underline a reference to the style key `PhoneTextNormalStyle` with a red squiggle. If that isn't corrected, then the app will immediately terminate when you try to deploy it to the emulator or device. So, it's important to attend to XAML markup correctness. And you will find Visual Studio to be a great tool for catching such issues.

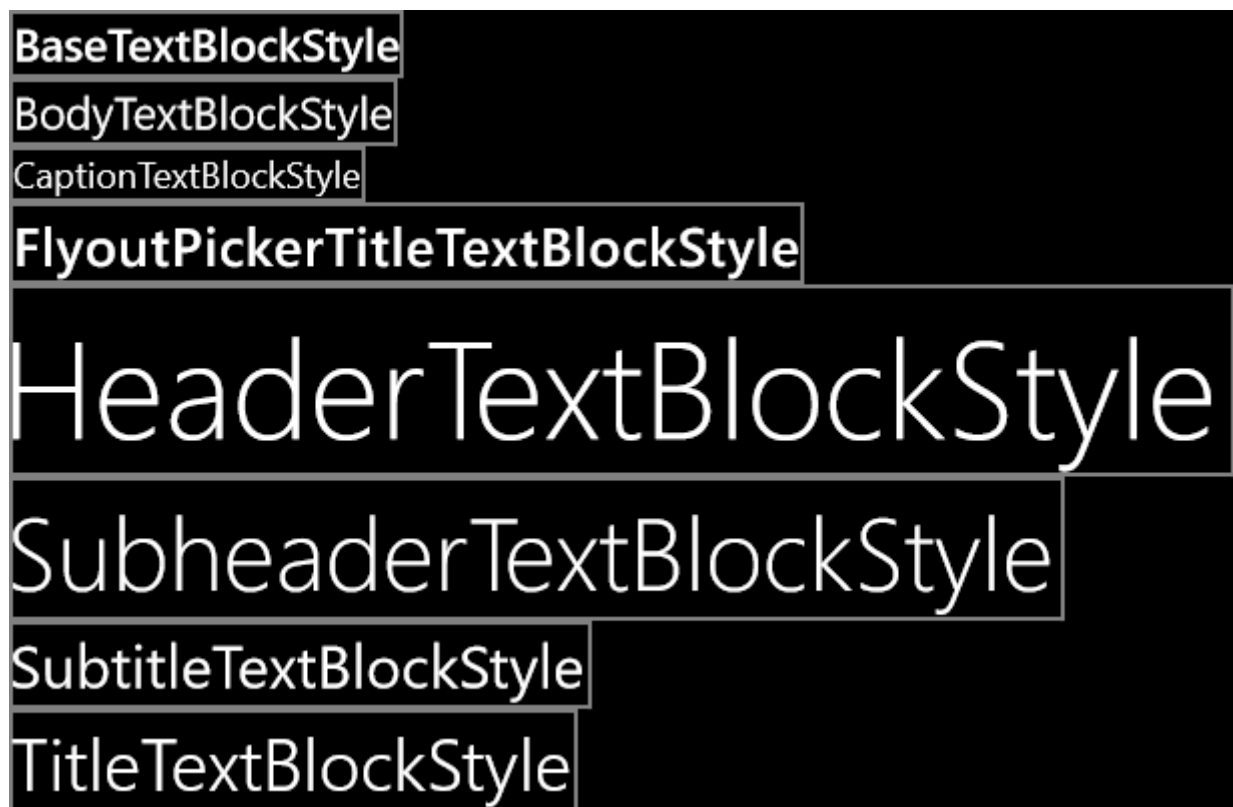
Also, see [Text](#), below.

Status bar (system tray)

The system tray (set in XAML markup with `shell:SystemTray.IsVisible`) is now called the status bar, and it is shown by default. You can control its visibility in imperative code by calling the [Windows.UI.ViewManagement.StatusBar.ShowAsync](#) and [HideAsync](#) methods.

Text

Text (or typography) is an important aspect of a UWP app and, while porting, you may want to revisit the visual designs of your views so that they are in harmony with the new design language. Use these illustrations to find the UWP **TextBlock** system styles that are available. Find the ones that correspond to the Windows Phone Silverlight styles you used. Alternatively, you can create your own universal styles and copy the properties from the Windows Phone Silverlight system styles into those.



System TextBlock styles for Windows 10 apps

In a Windows Phone Silverlight app, the default font family is Segoe WP. In a Windows 10 app, the default font family is Segoe UI. As a result, font metrics in your app may look different. If you want to reproduce the look of your Windows Phone Silverlight text, you can set your own metrics using properties such as [LineHeight](#) and [LineStackingStrategy](#). For more info, see [Guidelines for fonts](#) and [Design UWP apps](#).

Theme changes

For a Windows Phone Silverlight app, the default theme is dark by default. For Windows 10 devices, the default theme has changed, but you can control the theme used by declaring a requested theme in App.xaml. For example, to use a dark theme on all devices, add `RequestedTheme="Dark"` to the root Application element.

Tiles

Tiles for UWP apps have behaviors similar to Live Tiles for Windows Phone Silverlight apps, although there are some differences. For example, code that calls the `Microsoft.Phone.Shell.ShellTile.Create` method to create secondary tiles should be ported to call `SecondaryTile.RequestCreateAsync`. Here is a before-and-after example, first the Windows Phone Silverlight version:

C#

```
var tileData = new IconicTileData()
{
    Title = this.selectedBookSku.Title,
    WideContent1 = this.selectedBookSku.Title,
    WideContent2 = this.selectedBookSku.Author,
    SmallIconImage = this.SmallIconImageAsUri,
    IconImage = this.IconImageAsUri
};

ShellTile.Create(this.selectedBookSku.NavigationUri, tileData, true);
```

And the UWP equivalent:

C#

```
var tile = new SecondaryTile(
    this.selectedBookSku.Title.Replace(" ", string.Empty),
    this.selectedBookSku.Title,
    this.selectedBookSku.ArgumentString,
    this.IconImageAsUri,
    TileSize.Square150x150);

await tile.RequestCreateAsync();
```

Code that updates a tile with the `Microsoft.Phone.Shell.ShellTile.Update` method, or the `Microsoft.Phone.Shell.ShellTileSchedule` class, should be ported to use the `TileUpdateManager`, `TileUpdater`, `TileNotification`, and/or `ScheduledTileNotification` classes.

For more info on tiles, toasts, badges, banners, and notifications, see [Creating tiles](#) and [Working with tiles, badges, and toast notifications](#). For specifics about sizes of visual assets used for UWP Tiles, see [Tile and toast visual assets](#).

Toasts

Code that displays a toast with the **Microsoft.Phone.Shell.ShellToast** class should be ported to use the [ToastNotificationManager](#), [ToastNotifier](#), [ToastNotification](#), and/or [ScheduledToastNotification](#) classes. Note that on mobile devices, the consumer-facing term for "toast" is "banner".

See [Working with tiles, badges, and toast notifications](#).

View or effective pixels, viewing distance, and scale factors

Windows Phone Silverlight apps and Windows 10 apps differ in the way they abstract the size and layout of UI elements away from the actual physical size and resolution of devices. A Windows Phone Silverlight app uses view pixels to do this. With Windows 10, the concept of view pixels has been refined into that of effective pixels. Here's an explanation of that term, what it means, and the extra value it offers.

The term "resolution" refers to a measure of pixel density and not, as is commonly thought, pixel count. "Effective resolution" is the way the physical pixels that compose an image or glyph resolve to the eye given differences in viewing distance and the physical pixel size of the device (pixel density being the reciprocal of physical pixel size). Effective resolution is a good metric to build an experience around because it is user-centric. By understanding all the factors, and controlling the size of UI elements, you can make the user's experience a good one.

To a Windows Phone Silverlight app, all phone screens are exactly 480 view pixels wide, without exception, no matter how many physical pixels the screen has, nor what its pixel density or physical size is. This means that an **Image** element with `Width="48"` will be exactly one tenth of the width of the screen of any phone that can run the Windows Phone Silverlight app.

To a Windows 10 app, it is *not* the case that all devices are some fixed number of effective pixels wide. That's probably obvious, given the wide range of devices that a UWP app can run on. Different devices are a different number of effective pixels wide, ranging from 320 epx for the smallest devices, to 1024 epx for a modest-sized monitor, and far beyond to much higher widths. All you have to do is continue to use auto-sized

elements and dynamic layout panels as you always have. There will also be some cases where you'll set the properties of your UI elements to a fixed size in XAML markup. A scale factor is automatically applied to your app depending on what device it runs on and the display settings made by the user. And that scale factor serves to keep any UI element with a fixed size presenting a more-or-less constant-sized touch (and reading) target to the user across a wide variety of screen sizes. And together with dynamic layout your UI won't merely optically scale on different devices, it will instead do what's necessary to fit the appropriate amount of content into the available space.

Because 480 was formerly the fixed width in view pixels for a phone-sized screen, and that value is now typically smaller in effective pixels, a rule of thumb is to multiply any dimension in your Windows Phone Silverlight app markup by a factor of 0.8.

So that your app has the best experience across all displays, we recommend that you create each bitmap asset in a range of sizes, each suitable for a particular scale factor. Providing assets at 100%-scale, 200%-scale, and 400%-scale (in that priority order) will give you excellent results in most cases at all the intermediate scale factors.

Note If, for whatever reason, you cannot create assets in more than one size, then create 100%-scale assets. In Microsoft Visual Studio, the default project template for UWP apps provides branding assets (tile images and logos) in only one size, but they are not 100%-scale. When authoring assets for your own app, follow the guidance in this section and provide 100%, 200%, and 400% sizes, and use asset packs.

If you have intricate artwork, then you may want to provide your assets in even more sizes. If you're starting with vector art, then it's relatively easy to generate high-quality assets at any scale factor.

We don't recommend that you try to support all of the scale factors, but the full list of scale factors for Windows 10 apps is 100%, 125%, 150%, 200%, 250%, 300%, and 400%. If you provide them, the Store will pick the correct-sized asset(s) for each device, and only those assets will be downloaded. The Store selects the assets to download based on the DPI of the device.

For more info, see [Responsive design 101 for UWP apps](#).

Window size

In your UWP app, you can specify a minimum size (both width and height) with imperative code. The default minimum size is 500x320epx, and that's also the smallest minimum size accepted. The largest minimum size accepted is 500x500epx.

```
Windows.UI.ViewManagement.ApplicationView.GetForCurrentView().SetPreferredMinSize  
    (new Size { Width = 500, Height = 500 });
```

The next topic is [Porting for I/O, device, and app model](#).

Related topics

- [Namespace and class mappings](#)

Porting Windows Phone Silverlight to UWP for I/O, device, and app model

Article • 10/20/2022

The previous topic was [Porting XAML and UI](#).

Code that integrates with the device itself and its sensors involves input from, and output to, the user. It can also involve processing data. But, this code is not generally thought of as either the UI layer or the data layer. This code includes integration with the vibration controller, accelerometer, gyroscope, microphone and speaker (which intersect with speech recognition and synthesis), (geo)location, and input modalities such as touch, mouse, keyboard, and pen.

Application lifecycle (process lifetime management)

Your Windows Phone Silverlight app contains code to save and restore its application state and its view state in order to support being tombstoned and subsequently re-activated. The app lifecycle of Universal Windows Platform (UWP) apps has strong parallels with that of Windows Phone Silverlight apps, since they're both designed with the same goal of maximizing the resources available to whichever app the user has chosen to have in the foreground at any moment. You'll find that your code will adapt to the new system reasonable easily.

Note Pressing the hardware **Back** button automatically terminates a Windows Phone Silverlight app. Pressing the hardware **Back** button on a mobile device *does not* automatically terminate a UWP app. Instead, it becomes suspended, and then it may be terminated. But, those details are transparent to an app that responds appropriately to application lifecycle events.

A "debounce window" is the period of time between the app becoming inactive and the system raising the suspending event. For a UWP app, there is no debounce window; the suspension event is raised as soon as an app becomes inactive.

For more info, see [App lifecycle](#).

Camera

Windows Phone Silverlight camera capture code uses the `Microsoft.Devices.Camera`, `Microsoft.Devices.PhotoCamera`, or `Microsoft.Phone.Tasks.CameraCaptureTask` classes. To port that code to the Universal Windows Platform (UWP), you can use the [MediaCapture](#) class. There is a code example in the [CapturePhotoToStorageFileAsync](#) topic. That method allows you to capture a photo to a storage file, and it requires the `microphone` and `webcam` [device capabilities](#) to be set in the app package manifest.

Another option is the [CameraCaptureUI](#) class, which also requires the `microphone` and `webcam` [device capabilities](#).

Lens apps are not supported for UWP apps.

Detecting the platform your app is running on

The way of thinking about app-targeting changes with Windows 10. The new conceptual model is that an app targets the Universal Windows Platform (UWP) and runs across all Windows devices. It can then opt to light up features that are exclusive to particular device families. If needed, the app also has the option to limit itself to targeting one or more device families specifically. For more info on what device families are—and how to decide which device family to target—see [Guide to UWP apps](#).

Note We recommend that you not use operating system or device family to detect the presence of features. Identifying the current operating system or device family is usually not the best way to determine whether a particular operating system or device family feature is present. Rather than detecting the operating system or device family (and version number), test for the presence of the feature itself (see [Conditional compilation, and adaptive code](#)). If you must require a particular operating system or device family, be sure to use it as a minimum supported version, rather than design the test for that one version.

To tailor your app's UI to different devices, there are several techniques that we recommend. Continue to use auto-sized elements and dynamic layout panels as you always have. In your XAML markup, continue to use sizes in effective pixels (formerly view pixels) so that your UI adapts to different resolutions and scale factors (see [View/effective pixels, viewing distance, and scale factors](#)). And use Visual State Manager's adaptive triggers and setters to adapt your UI to the window size (see [Guide to UWP apps](#)).

However, if you have a scenario where it is unavoidable to detect the device family, then you can do that. In this example, we use the [AnalyticsVersionInfo](#) class to navigate to a page tailored for the mobile device family where appropriate, and we make sure to fall back to a default page otherwise.

C#

```
if (Windows.System.Profile.AnalyticsInfo.VersionInfo.DeviceFamily ==  
"Windows.Mobile")  
    rootFrame.Navigate(typeof(MainPageMobile), e.Arguments);  
else  
    rootFrame.Navigate(typeof(MainPage), e.Arguments);
```

Your app can also determine the device family that it is running on from the resource selection factors that are in effect. The example below shows how to do this imperatively, and the [ResourceContext.QualifierValues](#) topic describes the more typical use case for the class in loading device family-specific resources based on the device family factor.

C#

```
var qualifiers =  
Windows.ApplicationModel.Resources.Core.ResourceContext.GetForCurrentView().  
QualifierValues;  
string deviceFamilyName;  
bool isDeviceFamilyNameKnown = qualifiers.TryGetValue("DeviceFamily", out  
deviceFamilyName);
```

Also, see [Conditional compilation, and adaptive code](#).

Device status

A Windows Phone Silverlight app can use the **Microsoft.Phone.Info.DeviceStatus** class to get info about the device on which the app is running. While there is no direct UWP equivalent for the **Microsoft.Phone.Info** namespace, here are some properties and events that you can use in a UWP app in place of calls to members of the **DeviceStatus** class.

Windows Phone Silverlight	UWP
ApplicationCurrentMemoryUsage and ApplicationCurrentMemoryUsageLimit properties	MemoryManager.AppMemoryUsage and AppMemoryUsageLimit properties
ApplicationPeakMemoryUsage property	Use the memory profiling tools in Visual Studio. For more info, see Measure memory usage .
DeviceFirmwareVersion property	EasClientDeviceInformation.SystemFirmwareVersion property (desktop device family only)

Windows Phone Silverlight	UWP
DeviceHardwareVersion property	EasClientDeviceInformation.SystemHardwareVersion property (desktop device family only)
DeviceManufacturer property	EasClientDeviceInformation.SystemManufacturer property (desktop device family only)
DeviceName property	EasClientDeviceInformation.SystemProductName property (desktop device family only)
DeviceTotalMemory property	No equivalent
IsKeyboardDeployed property	No equivalent. This property provides information about hardware keyboards for mobile devices, which are not commonly used.
IsKeyboardPresent property	No equivalent. This property provides information about hardware keyboards for mobile devices, which are not commonly used.
KeyboardDeployedChanged event	No equivalent. This property provides information about hardware keyboards for mobile devices, which are not commonly used.
PowerSource property	No equivalent
PowerSourceChanged event	Handle the RemainingChargePercentChanged event (mobile device family only). The event is raised when the value of the RemainingChargePercent property (mobile device family only) decreases by 1%.

Location

When an app that declares the Location capability in its app package manifest runs on Windows 10, the system will prompt the end-user for consent. So, if your app displays its own custom consent prompt, or if it provides an on-off toggle, then you will want to remove that so that the end-user is only prompted once.

Orientation

The UWP app equivalent of the **PhoneApplicationPage.SupportedOrientations** and **Orientation** properties is the [uap:InitialRotationPreference](#) element in the app package manifest. Select the **Application** tab if it isn't already selected and select one or more check boxes under **Supported rotations** to record your preferences.

You're encouraged, however, to design the UI of your UWP app to look great regardless of device orientation and screen size. There's more about that in [Porting for form factors and user experience](#), which is the topic after next.

The next topic is [Porting business and data layers](#).

Porting Windows Phone Silverlight business and data layers to UWP

Article • 10/20/2022

The previous topic was [Porting for I/O, device, and app model](#).

Behind your UI are your business and data layers. The code in these layers calls operating system and .NET Framework APIs (for example, background processing, location, the camera, the file system, network, and other data access). The vast majority of those are [available to a Universal Windows Platform \(UWP\) app](#), so you can expect to be able to port much of this code without change.

Asynchronous methods

One of the priorities of the Universal Windows Platform (UWP) is to enable you to build apps that are truly, and consistently, responsive. Animations are always smooth, and touch interactions such as panning and swiping are instantaneous and free of lag, making it feel like the UI is glued to your finger. To achieve this, any UWP API that can't guarantee to complete within 50ms has been made asynchronous and its name suffixed with **Async**. Your UI thread will return immediately from calling an **Async** method, and the work will take place on another thread. Consuming an **Async** method is made very easy, syntactically, using the C# **await** operator, JavaScript promise objects, and C++ continuations. For more info, see [Asynchronous programming](#).

Background processing

A Windows Phone Silverlight app can use a managed **ScheduledTaskAgent** object to perform a task while the app is not in the foreground. A UWP app uses the [BackgroundTaskBuilder](#) class to create and register a background task in a similar way. You define a class that implements the work of your background task. The system runs your background task periodically, calling the [Run](#) method of your class to execute the work. In a UWP app, remember to set the **Background Tasks** declaration in the app package manifest. For more info, see [Support your app with background tasks](#).

To transfer large data files in the background, a Windows Phone Silverlight app uses the **BackgroundTransferService** class. A UWP app uses APIs in the [Windows.Networking.BackgroundTransfer](#) namespace to do this. The features use a similar pattern to initiate transfers, but the new API has improved capabilities and performance. For more info, see [Transferring data in the background](#).

A Windows Phone Silverlight app uses the managed classes in the **Microsoft.Phone.BackgroundAudio** namespace to play audio while the app is not in the foreground. The UWP uses the Windows Phone Store app model, see [Background Audio](#) and the [Background audio](#)  sample.

Cloud services, networking, and databases

Hosting data and app services in the cloud is possible using Azure. See [Getting Started with Mobile Services](#). For solutions that require both online and offline data see: [Using offline data sync in Mobile Services](#).

The UWP has partial support for the **System.Net.HttpWebRequest** class, but the **System.Net.WebClient** class is not supported. The recommended, forward-looking alternative is the **Windows.Web.Http.HttpClient** class (or **System.Net.Http.HttpClient** if you need your code to be portable to other platforms that support .NET). These APIs use **System.Net.Http.HttpRequestMessage** to represent an HTTP request.

UWP apps do not currently include built-in support for data-intensive scenarios such as line of business (LOB) scenarios. However, you can make use SQLite for local transactional database services. For more info, see [SQLite](#) .

Pass absolute URIs, not relative URIs, to Windows Runtime types. See [Passing a URI to the Windows Runtime](#).

Launchers and Choosers

With Launchers and Choosers (found in the **Microsoft.Phone.Tasks** namespace), a Windows Phone Silverlight app can interact with the operating system to perform common operations such as composing an email, choosing a photo, or sharing certain kinds of data with another app. Search for **Microsoft.Phone.Tasks** in the topic [Windows Phone Silverlight to Windows 10 namespace and class mappings](#) to find the equivalent UWP type. These range from similar mechanisms, called launchers and pickers, to implementing a contract for sharing data between apps.

A Windows Phone Silverlight app can be put into a dormant state or even tombstoned when using, for example, the photo Chooser task. A UWP app remains active and running while using the **FileOpenPicker** class.

Monetization (trial mode and in-app purchases)

A Windows Phone Silverlight app can use the UWP [CurrentApp](#) class for most of its trial mode and in-app purchase functionality, so that code doesn't need to be ported. But, a Windows Phone Silverlight app calls **MarketplaceDetailTask.Show** to offer the app for purchase:

C#

```
private void Buy()
{
    MarketplaceDetailTask marketplaceDetailTask = new
MarketplaceDetailTask();
    marketplaceDetailTask.ContentType =
MarketplaceContentType.Applications;
    marketplaceDetailTask.Show();
}
```

Port that code to call the UWP [RequestAppPurchaseAsync](#) method:

C#

```
private async void Buy()
{
    await
Windows.ApplicationModel.Store.CurrentApp.RequestAppPurchaseAsync(false);
}
```

If you have code that simulates your app purchase and in-app purchase features for testing purposes, then you can port that to use the [CurrentAppSimulator](#) class instead.

Notifications for tile or toast updates

Notifications are an extension of the push notification model for Windows Phone Silverlight apps. When you receive a notification from the Windows Push Notification Service (WNS), you can surface the info to the UI with a tile update or with a toast. For porting the UI side of your notification features, see [Tiles and toasts](#).

For more details on the use of notifications in a UWP app, see [Sending toast notifications](#).

For info and tutorials on using tiles, toasts, badges, banners, and notifications in a Windows Runtime app using C++, C#, or Visual Basic, see [Working with tiles, badges, and toast notifications](#).

Storage (file access)